

Lecture Notes

Numerical Optimization

Winter Term 2025/2026

Michael C. Lehn

Institute of Numerical Mathematics
Ulm University

Contents

1	Introduction	4
2	Nonlinear Least Squares Problems	5
§1	Problem Statement	5
§2	Test Problems	10
§3	Derivatives of the Objective Function	14
§4	Newton–Raphson Method	17
§5	Gauss–Newton	20
§6	Outlook: Levenberg–Marquardt method	23
3	Unconstrained optimization	24
§1	Optimality conditions and convexity	25
§2	Direct Search Methods	32
§2.1	Hooke–Jeeves Method	33
§2.2	Nelder–Mead Method	34
§2.3	Numerical Experiments: Rosenbrock Function	37
§3	Descent Methods with Line Search	38
§3.1	Basic Concepts	39
§3.2	Line Search and Armijo Rule	40
§3.3	Global Convergence Statement for Gradient Descent with Armijo Step Size	41
§3.4	Efficient Step Size Strategies	45
§3.5	Non-linear conjugate gradient (cg) method	47
§4	Trust-Region Methods	49
A	Supplementary Notes to Chapters	50
§1	Least Square	50

List of Algorithms

1	Gauss–Newton algorithm	22
2	Hooke–Jeeves method	34
3	Exploration procedure	34
4	Nelder–Mead method (in the notation of the subsection)	37
5	Gradient Descent with Armijo Step Size	42
6	Descent Method with Step Size Strategy T	45
7	Linear cg-algorithm	48
8	Non-linear cg-algorithm after Fletcher–Reeves	48

Listings

2.1	Function <code>create_circ_data</code> : Generate synthetic circle data.	13
2.2	Function <code>draw_circle</code> : Visualize data and fitted circles.	13

Chapter 1 Introduction

The problems considered in this chapter arise in practical applications where measurements are affected by noise and models are nonlinear. In general, such problems are not solvable in a strict mathematical sense. However, they do not arise arbitrarily but have a specific structure and origin that can and must be exploited.

Numerical mathematics provides the framework to deal with such situations. By using approximation, iteration, and careful analysis, it becomes possible to obtain practically useful results even when an exact solution cannot be found.

This is not a compromise of mathematics—it is its frontier. Numerical mathematics is, in this sense, the spearhead of mathematics.

Chapter 2 Nonlinear Least Squares Problems

In the course on Numerical Linear Algebra, we studied the *linear* least squares problem, that is, minimization problems of the form

$$\|Ax - b\|_2^2 \rightarrow \min$$

for a full-rank matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$ with $m \geq n$. Defining

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = \|Ax - b\|_2^2,$$

we saw that

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x) \iff \nabla f(\hat{x}) = \mathbf{0}.$$

Because f is convex in x , this necessary condition is also sufficient for a global minimum. Since

$$\nabla f(x) = 2A^T(Ax - b),$$

the minimizer $\hat{x}$ satisfies the so-called *normal equations*

$$A^T A x = A^T b.$$

In the Numerical Linear Algebra course, the emphasis was on algorithms suitable for solving these equations efficiently and stably. For example, the Cholesky decomposition exploits the fact that $A^T A$ is symmetric and positive definite, while the QR decomposition allows one to compute the solution without explicitly forming $A^T A$, which improves numerical stability.

In the following, we will generalize the problem by allowing the objective function f to be nonlinear in x . This is highly relevant in practice, since many interesting problems fall into this category. From a theoretical perspective, however, nonlinear least squares problems are considerably more challenging.

§1 Problem Statement

In this chapter, we consider objective functions

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

of the special form

$$f(x) = \frac{1}{2} \sum_{i=1}^m F_i(x)^2,$$

where the functions $F_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $1 \leq i \leq m$, are assumed to be twice continuously differentiable. Equivalently, we can write

$$f(x) = \frac{1}{2} F(x)^T F(x) = \frac{1}{2} \|F(x)\|_2^2, \quad F(x) = \begin{pmatrix} F_1(x) \\ \vdots \\ F_m(x) \end{pmatrix}.$$

To facilitate later reference, we summarize this specific class of problems in the following definition.

Definition 2.1.1 (Least Squares Problem)

Let $F_1, \dots, F_m \in C^2(\mathbb{R}^n; \mathbb{R})$ with $m, n \geq 1$, and define

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = \frac{1}{2} \sum_{i=1}^m F_i(x)^2.$$

A point $\hat{x} \in \mathbb{R}^n$ is called a (global) solution of the *least squares problem* if

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x).$$

□

Having defined the general problem, we now verify that it indeed represents a genuine generalization of the least squares problem encountered in the course on Numerical Linear Algebra.

Example 2.1.2 (Linear least squares as a special case)

Let $a_1, \dots, a_m \in \mathbb{R}^n$ and $b_1, \dots, b_m \in \mathbb{R}$, and define

$$F_i(x) = a_i^T x - b_i, \quad 1 \leq i \leq m.$$

Then

$$\begin{aligned} f(x) &= \frac{1}{2} \sum_{i=1}^m (a_i^T x - b_i)^2 \\ &= \frac{1}{2} \|Ax - b\|_2^2, \end{aligned}$$

where

$$A = \begin{pmatrix} a_1^T \\ \vdots \\ a_m^T \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}.$$

Clearly,

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x) \iff \|A\hat{x} - b\|_2^2 = \min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2.$$

Thus, this special case is equivalent to the familiar *linear least squares problem*. If $m \geq n$ and A has full column rank, the minimizer is unique.

Hence, the nonlinear least squares problem truly generalizes the classical linear one: the objective function differs only by the factor $\frac{1}{2}$, which is introduced merely for convenience, as it simplifies the expression for the necessary condition of optimality (see (2.1) below). □

Despite the rather strong assumption that the objective function f is twice continuously differentiable and, by construction, bounded below by 0, no general statement can be made about the existence (or even uniqueness) of a solution.

Example 2.1.3

Let $m = n = 1$.

(a) For $F(x) = F_1(x) = 1 - x^2$ we have

$$f(x) = \frac{1}{2}F(x)^2 = \frac{1}{2}(1 - x^2)^2.$$

Clearly, $f(x) = 0$ if and only if $x = \pm 1$. Hence, a solution exists, but it is not unique.

(b) For $F(x) = F_1(x) = \frac{1}{1+x^2}$ we obtain

$$f(x) = \frac{1}{2(1+x^2)^2} > 0 \quad \text{for all } x \in \mathbb{R}.$$

Since

$$\lim_{x \rightarrow \infty} f(x) = 0,$$

no global minimizer exists.

□

If a solution $\hat{x}$ exists, that is, if the global minimum of f is attained, then $\hat{x}$ is in particular also a local minimum. A necessary condition for a local minimum (or, more generally, for a stationary point) is that the gradient vanishes:

$$f'(\hat{x}) = \nabla f(\hat{x})^T = \mathbf{0}^T. \quad (2.1)$$

A sufficient condition for a local minimum is that

$$\begin{aligned} \nabla f(\hat{x}) &= \mathbf{0} \quad \text{and} \quad r^T H_f(\hat{x}) r > 0 \quad \forall r \neq \mathbf{0} \\ \Rightarrow \exists \varepsilon > 0 : f(x) &> f(\hat{x}) \quad \forall x : \|x - \hat{x}\|_2 < \varepsilon. \end{aligned} \quad (2.2)$$

For completeness, these well-known results from analysis are reviewed in Appendix §1. That appendix also summarizes the notation and conventions used throughout this text, such as ∇f for the gradient (the $n \times 1$ Jacobian matrix) and H_f for the Hessian matrix.

However, the sufficient condition for a local minimum given in (2.2) is not always conclusive, as the following example shows.

Example 2.1.4 (Stationary points and higher derivatives)

Let $m = n = 1$.

(a) For $F_1(x) = x^2$ we have

$$f(x) = \frac{1}{2}F_1(x)^2 = \frac{1}{2}x^4,$$

and $\hat{x} = 0$ is clearly the point of the global minimum. Here

$$f'(0) = f''(0) = f^{(3)}(0) = 0, \quad f^{(4)}(0) = 4! > 0.$$

In general, if f is $2k$ -times continuously differentiable and

$$f'(\hat{x}) = f''(\hat{x}) = \cdots = f^{(2k-1)}(\hat{x}) = 0, \quad f^{(2k)}(\hat{x}) > 0,$$

then $\hat{x}$ is a local minimizer.

For $n > 1$, $f''(\hat{x})$ is a matrix (the Hessian). Checking its definiteness is numerically feasible, as known from the course on Numerical Linear Algebra. We also know the computational cost associated with this task. Higher derivatives (3rd, 4th, etc.) would be tensors. In principle, similar sufficient conditions could be formulated using them, but the computational effort grows rapidly. Since we only assume $f \in C^2(\mathbb{R}^n)$, we will not pursue this idea further, as higher derivatives may not even exist.

- (b) Even if we allow f to be infinitely often differentiable, examining higher derivatives alone may not suffice. Consider the classical example

$$F_1(x) = \begin{cases} e^{-1/x^2}, & x \neq 0, \\ 0, & x = 0. \end{cases}$$

Then F_1 is infinitely differentiable, and $x = 0$ is a global minimum of F_1 , but

$$F_1^{(k)}(0) = 0 \quad \text{for all } k \in \mathbb{N}.$$

The same holds for

$$f(x) = \frac{1}{2}F_1(x)^2,$$

where all derivatives vanish at $x = 0$, even though $x = 0$ is a strict global minimizer. Thus, higher-order derivatives alone do not necessarily provide sufficient information to identify local minima.

□

For a numerical method to be considered feasible, it is therefore reasonable to require merely that it produces stationary points of the objective function. Whether a stationary point corresponds to a local minimum can then be examined separately.

However, even this task is far from trivial. A conceivable strategy for finding the solution would be to determine all local minima and then select the one with the smallest function value. For such an approach to be theoretically feasible, it must at least be guaranteed that there exist only finitely many local minima.

To illustrate what “not feasible” may look like in practice and to appreciate the inherent difficulty of the problem, consider the following example.

Example 2.1.5 (Infinitely many non-global local minima)

Let

$$f(x) = \sin^2(x) + \sin^2(x^2).$$

Then $f(0) = 0$ and $f(x) > 0$ for all $x \neq 0$, so $\hat{x} = 0$ is the unique global minimizer. Nevertheless, f possesses infinitely many local minima. This can already be seen intuitively from the graph of f (see Figure 2.1), whose oscillations become increasingly dense for large values of $|x|$.

A proof of this statement is given at the end of this section in Remark 2.1.6. Before that, we discuss what such examples imply for numerical treatment.

□

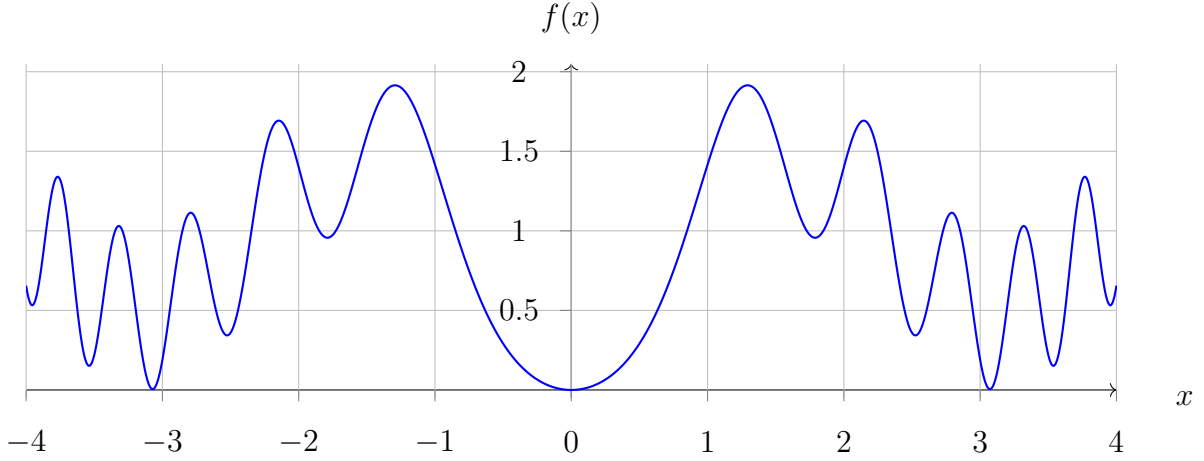


Figure 2.1: Graph of $f(x) = \sin^2(x) + \sin^2(x^2)$ on $[-4, 4]$.

In situations like this, a numerical optimization method can at best be expected to generate a sequence of local minimizers converging towards the global one. In general, however, the local minimizers cannot be determined analytically but only numerically. Hence, even finding a single local minimizer requires a numerical method that, under suitable assumptions, produces a sequence converging to a local stationary point. Whether such a stationary point corresponds to a local minimum, a saddle point, or a local maximum can, in general, only be determined *a posteriori*.

One should therefore not expect numerical optimization to provide a magical black box capable of solving such problems automatically. After this somewhat sobering realization, it is worth emphasizing that there are still good reasons to be optimistic: objective functions arising in applications do not fall from the sky. In practice, we usually know much more about them than merely that they are twice continuously differentiable. What we can develop is therefore not a single universal algorithm that solves all problems, but rather a toolbox of methods that—under appropriate assumptions—can handle specific subclasses of problems efficiently. Using this toolbox effectively requires understanding the underlying methods and their limitations, in particular the conditions under which they are guaranteed to work.

Before we turn to an application example in the next section, we give here the proof promised in Example 2.1.5.

Remark 2.1.6 (Proof of Example 2.1.5)

We consider

$$f(x) = \sin^2 x + \sin^2 x^2,$$

with

$$\begin{aligned} f'(x) &= 2 \sin x \cdot \cos x + 2 \sin x^2 \cdot \cos x^2 \cdot 2x \\ &= \sin 2x + 2x \sin 2x^2. \end{aligned}$$

We show the following three statements:

- (a) $x = 0$ is a global minimizer.

Since both terms in $f(x)$ are nonnegative and vanish simultaneously only for $x = 0$,

we have

$$f(x) \geq 0 \quad \text{for all } x \in \mathbb{R}, \quad f(0) = 0.$$

Hence $\hat{x} = 0$ is a global minimizer with $f(\hat{x}) = 0$.

(b) $x = 0$ is the unique global minimizer.

The equality $f(x) = 0$ requires $\sin x = 0$ and $\sin x^2 = 0$ simultaneously. This means

$$x = k\pi \quad \text{and} \quad x^2 = n\pi \quad \text{for some } k, n \in \mathbb{Z}.$$

Combining both gives $k^2\pi^2 = n\pi$, i.e. $n = k^2\pi$. As π is irrational, this is impossible for integer n, k unless $k = n = 0$. Therefore $x = 0$ is the only point where $f(x) = 0$, and thus the unique global minimizer.

(c) f has infinitely many local minima.

Define

$$a_n = \sqrt{\frac{\pi}{4}(4n+1)}, \quad b_n = \sqrt{\frac{\pi}{4}(4n+3)}, \quad n \in \mathbb{N}.$$

Then

$$\sin 2a_n^2 = 1, \quad \sin 2b_n^2 = -1.$$

Because

$$b_n > a_n \geq \sqrt{\frac{\pi}{4}} > \frac{1}{2},$$

we have

$$\begin{aligned} f'(a_n) &= \sin 2a_n + 2a_n > \sin 2a_n + 1 \geq 0, \\ f'(b_n) &= \sin 2b_n - 2b_n < \sin 2b_n - 1 \leq 0. \end{aligned}$$

And since $a_n < b_n < a_{n+1}$, the intermediate value theorem ensures at least one zero of f' in each of the intervals (a_n, b_n) and (b_n, a_{n+1}) . Hence f' changes sign infinitely often, and f possesses infinitely many local minima and maxima.

In particular, $x = 0$ is the unique global minimizer, while all other local minima satisfy $f(x) > 0$. □

§2 Test Problems

In the following, three related problems are introduced that will later serve as test cases for numerical methods designed to solve least squares problems. Each problem is constructed such that the first and second derivatives of the objective function can be computed analytically — which, of course, is not generally possible in practical applications.

In all three cases, we are given points $(x_i, y_i)^T \in \mathbb{R}^2$ for $1 \leq i \leq m$, which are assumed to lie approximately on the boundary of a circle with true center $(\bar{c}_x, \bar{c}_y)^T$ and true radius $\bar{r}$, according to an underlying model, but are affected by stochastic noise.

Formally, each observed data point can be expressed as

$$\begin{aligned} x_i &= \bar{x}_i + \varepsilon_{x,i}, \\ y_i &= \bar{y}_i + \varepsilon_{y,i}, \end{aligned} \quad 1 \leq i \leq m,$$

where the ideal (noise-free) points $(\bar{x}_i, \bar{y}_i)$ are located on the circumference of a circle with true parameters $(\bar{c}_x, \bar{c}_y, \bar{r})$ and satisfy

$$\begin{aligned}\bar{x}_i &= \bar{c}_x + \bar{r} \cos \varphi_i, \\ \bar{y}_i &= \bar{c}_y + \bar{r} \sin \varphi_i,\end{aligned}\quad 1 \leq i \leq m.$$

The angles φ_i are independently and uniformly distributed as

$$\varphi_i \stackrel{\text{iid}}{\sim} \mathcal{U}(0, 2\pi),$$

and the noise terms are independent and identically distributed as

$$\varepsilon_{x,i}, \varepsilon_{y,i} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

For a single data point $(x_i, y_i)^T$ we define the residual

$$F_i(r, c_x, c_y) = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2} - r, \quad (2.3)$$

and the least-squares objective

$$f(r, c_x, c_y) = \frac{1}{2} \sum_{i=1}^m F_i(r, c_x, c_y)^2. \quad (2.4)$$

If the data are noise-free (formally, $\sigma = 0$), we have

$$f(\bar{r}, \bar{c}_x, \bar{c}_y) = 0.$$

For noisy data with $\sigma > 0$, this equality no longer holds exactly. The expected value of $f(\bar{r}, \bar{c}_x, \bar{c}_y)$ is strictly positive due to the bias introduced by the noise. Nevertheless, for small noise levels, the global minimizer of f will typically be close to the true parameters $\bar{r}, \bar{c}_x$, and $\bar{c}_y$, and can therefore serve as a useful estimator of the unknown true parameters.

We now consider the following three test problems, with objectives $f_{\text{ex1}}, f_{\text{ex2}}, f_{\text{ex3}}$.

- **Example 1 (known center at the origin)**

The center of the circle is known to be at $(c_x, c_y) = (0, 0)$, while the radius r is unknown. Although negative values of r have no physical meaning, we deliberately allow $r < 0$ so that the problem remains unconstrained. This leads to

$$f_{\text{ex1}}: \mathbb{R} \rightarrow \mathbb{R}, \quad f_{\text{ex1}}(r) := f(r, 0, 0). \quad (2.5)$$

A particular advantage of this setting is that both f and its derivative f' can be easily visualized.

- **Example 2 (partially known center)**

The circle center is partially known: $c_y = 0$ is fixed, whereas c_x and r are unknown. This leads to

$$f_{\text{ex2}}: \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f_{\text{ex2}}(r, c_x) := f(r, c_x, 0). \quad (2.6)$$

In this case, a contour plot of $f(c_x, r)$ can be generated, giving intuitive insight into the structure of the objective function and the distribution of its extrema.

- **Example 3 (unknown center and radius)**

The most general case, where r , c_x , and c_y are all unknown. This leads to

$$f_{\text{ex3}}: \mathbb{R}^3 \rightarrow \mathbb{R}, \quad f_{\text{ex3}}(r, c_x, c_y) := f(r, c_x, c_y). \quad (2.7)$$

Here no parameters are fixed, making the problem fully nonlinear and three-dimensional.

[Listing 2.1](#) shows the implementation of the function

```
function [x, y] = create_circ_data(m, r_, cx_, cy_, sigma)
```

which generates synthetic test data for the problems described above. Its parameters have the following meaning:

- **m** – number of data points,
- **r_** – true radius, corresponding to $\bar{r}$,
- **cx_**, **cy_** – true circle center, corresponding to $\bar{c}_x$ and $\bar{c}_y$,
- **sigma** – standard deviation of the normally distributed noise.

The function returns two column vectors

$$x = (x_1, \dots, x_m)^T, \quad y = (y_1, \dots, y_m)^T,$$

representing the observed data points.

To build intuition for the three test problems, it is instructive to visualize the noisy data together with the “true” and “estimated” circles. [Listing 2.2](#) shows the implementation of the function

```
function draw_circle(x, y, r_, cx_, cy_, r, cx, cy)
```

Its arguments are:

- **x**, **y** – data vectors as returned by `create_circ_data`,
- **r_**, **cx_**, **cy_** – true circle parameters $(\bar{r}, \bar{c}_x, \bar{c}_y)$,
- **r**, **cx**, **cy** – estimated circle parameters.

The function plots the data points (x_i, y_i) , the true circle (in blue, with radius $\bar{r}$ and center $(\bar{c}_x, \bar{c}_y)$), and the estimated circle (in red, with radius r and center (c_x, c_y)). This visualization provides an immediate geometric impression of how well the estimated parameters fit the noisy data.

[Figure 2.2](#) illustrates characteristic aspects of the test problems introduced above. The three subfigures do not visualize all cases (f_{ex3} would be three-dimensional and is therefore difficult to display), but they provide geometric intuition for the first two examples and for the general nature of the least squares objectives.

1. [Subfigure 2.2a](#) shows how the noisy data points are scattered around the “true” circle. The data were generated with `[x, y] = create_circ_data(500, 10, 0, 0, 1)` and visualized using `draw_circle(x, y, 10, 0, 0, 10, 0, 0)`.

Listing 2.1: *Function create_circ_data: Generate synthetic circle data.*

```
1 function [x, y] = create_circ_data(m, r_, cx_, cy_, sigma)
2
3     arguments
4         m (1,1) double {mustBeInteger, mustBeNonnegative}
5         r_ (1,1) double {mustBeFinite}
6         cx_ (1,1) double {mustBeFinite}
7         cy_ (1,1) double {mustBeFinite}
8         sigma (1,1) double {mustBeNonnegative} = 1
9     end
10
11     if m == 0
12         x = zeros(0,1);
13         y = zeros(0,1);
14         return
15     end
16
17     phi = 2 * pi * rand(m,1);
18     noise = sigma * randn(m,2);
19
20     x = cx_ + r_ * cos(phi) + noise(:, 1);
21     y = cy_ + r_ * sin(phi) + noise(:, 2);
22 end
```

Listing 2.2: *Function draw_circle: Visualize data and fitted circles.*

```
1 function draw_circle(x, y, r_, cx_, cy_, r, cx, cy)
2
3     plot(x, y, 'bx');
4     hold on; axis equal; grid on
5
6     theta = linspace(0, 2 * pi, 400);
7     plot(cx_ + r*cos(theta), cy_ + r * sin(theta), ...
8         'b-', 'LineWidth', 1.2);
9
10    theta = linspace(0, 2 * pi, 400);
11    plot(cx + r_est*cos(theta), cy + r * sin(theta), ...
12        'r-', 'LineWidth', 1.2);
13
14    xlabel('x'), ylabel('y')
15    title('Noisy circle data: r = 10, (c_x, c_y) = (0, 0)');
16
17    hold off
18 end
```

2. [Subfigure 2.2b](#) displays the objective $f_{\text{ex1}}(r)$ for $r \in [9.5, 10.5]$. The function is quadratic and therefore convex in r , which can also be shown analytically. The minimum is slightly shifted from $r = 10$, reflecting the influence of the measurement noise.
3. [Subfigure 2.2c](#) presents a contour plot of $f_{\text{ex2}}(r, c_x)$ for $r \in [0, 30]$ and $c_x \in [0, 30]$. The contour lines reveal that f_{ex2} is not convex (if it were, all level sets would be convex as well). This nonconvexity can also be established formally. For f_{ex3} , visualization would require plotting two-dimensional manifolds in $\mathbb{R}^3$, which is considerably more challenging.

§3 Derivatives of the Objective Function

In order to formulate numerical methods such as Newton–Raphson, Gauss–Newton, or Levenberg–Marquardt, we require the first and second derivatives of the objective function f . The goal is to express the partial derivatives of f in terms of the derivatives of the residuals $F_1, \dots, F_m$, so that the resulting expressions can be easily adapted to specific least-squares problems.

We use i and j as row and column indices, respectively, and denote the summation index by ℓ . Let

$$f(x) = \frac{1}{2} \sum_{\ell=1}^m F_\ell(x)^2, \quad x \in \mathbb{R}^n.$$

For the partial derivatives of f , the chain rule gives

$$\frac{\partial f}{\partial x_j}(x) = \frac{1}{2} \sum_{\ell=1}^m 2 F_\ell(x) \frac{\partial F_\ell}{\partial x_j}(x) = \sum_{\ell=1}^m F_\ell(x) \frac{\partial F_\ell}{\partial x_j}(x). \quad (2.8)$$

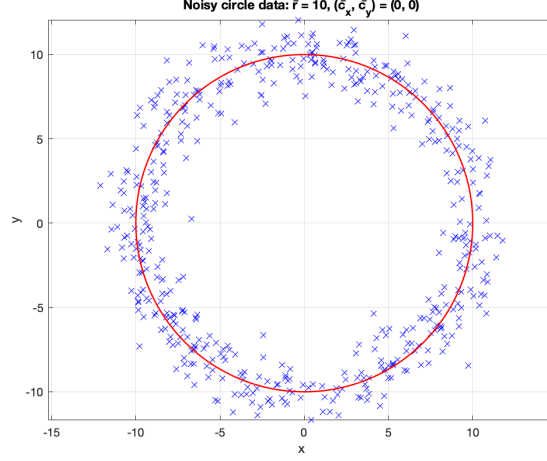
Hence, the gradient is

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix} = \sum_{\ell=1}^m F_\ell(x) \nabla F_\ell(x) \quad \text{with} \quad \nabla F_\ell(x) := \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) \\ \vdots \\ \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix}. \quad (2.9)$$

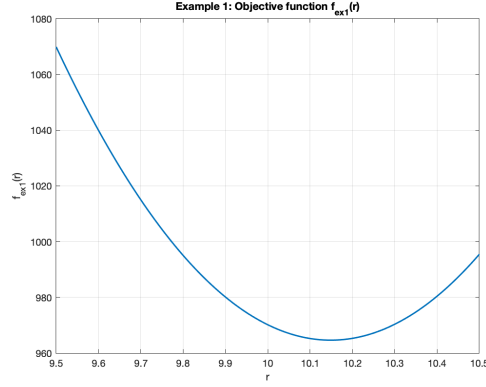
For the second partial derivatives, applying the product rule yields

$$\frac{\partial^2 f}{\partial x_i \partial x_j}(x) = \sum_{\ell=1}^m \left(\frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x) + F_\ell(x) \frac{\partial^2 F_\ell}{\partial x_i \partial x_j}(x) \right). \quad (2.10)$$

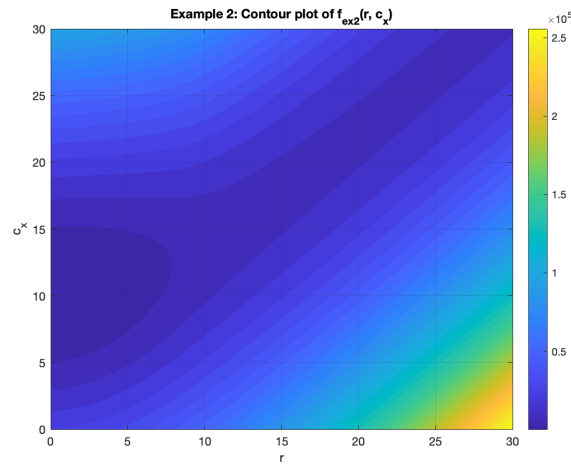
This is the entry of the Hessian matrix $H_f(x)$ in row i and column j . To write this more compactly, it is helpful to recognize how the collection of all products $\frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x)$ can



(a) Generated data using $[x, y] = \text{create_circ_data}(500, 10, 0, 0, 1)$ and visualized with $\text{draw_circle}(x, y, 10, 0, 0, 10, 0, 0)$. The noisy points (black) are distributed around the true circle (blue).



(b) Plot of $f_{\text{ex1}}(r)$ for $r \in [9.5, 10.5]$. The objective is convex in r , though the minimum is slightly shifted from the true radius $r = 10$ due to noise.



(c) Contour plot of $f_{\text{ex2}}(r, c_x)$ for $r \in [0, 30]$ and $c_x \in [0, 30]$. The objective is non-convex, showing multiple local minima.

Figure 2.2: Illustrations of the test problems. (a) Generated circle data with noise, (b) convex objective $f_{\text{ex1}}(r)$, and (c) non-convex objective $f_{\text{ex2}}(r, c_x)$.

be represented as a single matrix expression. Consider

$$\begin{aligned}
(\nabla F_1 \quad \dots \quad \nabla F_m) \begin{pmatrix} \nabla F_1^\top \\ \vdots \\ \nabla F_m^\top \end{pmatrix} &= \sum_{\ell=1}^m \nabla F_\ell \nabla F_\ell^\top = \sum_{\ell=1}^m \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) \\ \vdots \\ \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix} \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) & \dots & \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix} \\
&= \sum_{\ell=1}^m \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) \frac{\partial F_\ell}{\partial x_1}(x) & \dots & \frac{\partial F_\ell}{\partial x_1}(x) \frac{\partial F_\ell}{\partial x_n}(x) \\ \vdots & & \vdots \\ \frac{\partial F_\ell}{\partial x_n}(x) \frac{\partial F_\ell}{\partial x_1}(x) & \dots & \frac{\partial F_\ell}{\partial x_n}(x) \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix} \\
&= \begin{pmatrix} \vdots & & \vdots \\ \dots & \sum_{\ell=1}^m \frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x) & \dots \\ \vdots & & \vdots \end{pmatrix}_{1 \leq i, j \leq n}.
\end{aligned}$$

Thus, the (i, j) -entry of this matrix is precisely $\sum_{\ell=1}^m \frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x)$, which corresponds exactly to the first term in (2.10).

The Hessian of a single residual function F_ℓ is

$$H_{F_\ell}(x) = \left(\frac{\partial^2 F_\ell}{\partial x_i \partial x_j}(x) \right)_{1 \leq i, j \leq n}.$$

Combining these results with (2.10), we obtain the compact form

$$H_f(x) = \sum_{\ell=1}^m \left(\nabla F_\ell(x) \nabla F_\ell(x)^T + F_\ell(x) H_{F_\ell}(x) \right). \quad (2.11)$$

Jacobian notation (for later reference). Let

$$F: \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad F(x) = \begin{pmatrix} F_1(x) \\ \vdots \\ F_m(x) \end{pmatrix}.$$

For the least-squares problem introduced in Definition 2.1.1, the mapping F is assumed to be twice continuously differentiable. Its derivative $F'(x)$ is the *Jacobian matrix* (or Jacobian)

$$F'(x) = J(x) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1}(x) & \dots & \frac{\partial F_1}{\partial x_n}(x) \\ \vdots & & \vdots \\ \frac{\partial F_m}{\partial x_1}(x) & \dots & \frac{\partial F_m}{\partial x_n}(x) \end{pmatrix} = \begin{pmatrix} \nabla F_1(x)^T \\ \vdots \\ \nabla F_m(x)^T \end{pmatrix} \in \mathbb{R}^{m \times n}. \quad (2.12)$$

With this notation, the results obtained above can be expressed compactly in matrix form. From (2.9), the gradient of f can be written as

$$\nabla f(x) = J(x)^T F(x), \quad (2.13)$$

and from (2.11), the Hessian follows as

$$H_f(x) = J(x)^T J(x) + \sum_{\ell=1}^m F_\ell(x) H_{F_\ell}(x). \quad (2.14)$$

The first term $J(x)^T J(x)$ corresponds to $\sum_{\ell=1}^m \nabla F_\ell(x) \nabla F_\ell(x)^T$ and is often referred to as the *Gauss–Newton term*. The second term collects all contributions involving the Hessians $H_{F_\ell}(x)$ of the residuals F_ℓ . It vanishes when the residuals are small, which will motivate the *Gauss–Newton approximation*.

§4 Newton–Raphson Method

The idea is to determine stationary points $\hat{x}$, that is, points for which

$$\nabla f(\hat{x}) = \mathbf{0}.$$

As already mentioned, one may then examine whether such points correspond to local minima.

Define

$$g : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad g(x) = \nabla f(x).$$

Finding stationary points is therefore equivalent to solving a nonlinear system of equations, a problem known from the course *Numerical Analysis*. Since f is twice continuously differentiable, g is continuously differentiable as well. Hence, the Newton method is a natural candidate for computing such points:

$$x_{k+1} = x_k - g'(x_k)^{-1}g(x_k), \quad \text{starting from some } x_0.$$

For the method to be applicable, $g'(x_k)$ must be invertible at every step. Convergence can only be guaranteed under additional assumptions. Under certain conditions, the convergence is even quadratic if the initial guess is sufficiently close to the solution.

For reference, the following theorem was proven in the lecture.

Theorem 2.4.1 (Quadratic convergence of the Newton method for systems)

Let $M \subseteq \mathbb{R}^n$ be open and convex, $\|\cdot\|$ a norm in $\mathbb{R}^n$, $\|\cdot\|$ a compatible matrix norm in $\mathbb{R}^{n \times n}$, and $g : M \rightarrow \mathbb{R}^n$ a continuously differentiable function whose Jacobian $g'(x)$ is invertible for all $x \in M$, such that

$$\|(g'(x))^{-1}\| \leq \beta \quad \forall x \in M. \quad (2.15)$$

Furthermore, assume that $g'(x)$ is Lipschitz continuous on M with constant γ , i.e.

$$\|g'(x) - g'(y)\| \leq \gamma \|x - y\| \quad \forall x, y \in M. \quad (2.16)$$

Suppose there exists a solution $\hat{x} \in M$ of $g(x) = \mathbf{0}$. The initial guess x_0 satisfies

$$x_0 \in U_\omega := \{x \in \mathbb{R}^n : \|x - \hat{x}\| < \omega\},$$

where ω is sufficiently small such that $U_\omega \subseteq M$, and there exists a constant α satisfying

$$\frac{\beta\gamma\omega}{2} \leq \alpha < 1. \quad (2.17)$$

Then the Newton method is well defined and converges quadratically to $\hat{x}$.

The assumptions of the theorem are often difficult to verify in practice, and the behavior of the Newton method strongly depends on the specific problem at hand. To illustrate its application and to gain some intuition, we now consider the test problems introduced in Section §2.

Example 2.4.2 (Application to f_{ex1})

We first consider the simplest case,

$$f_{\text{ex1}}(r) := f(r, 0, 0),$$

that is, we assume the circle is centered at the origin and only the radius r is unknown. Hence, the center coordinates c_x and c_y are fixed, and all partial derivatives of F_i with respect to c_x and c_y vanish. The residuals simplify to

$$F_i(r) = \sqrt{x_i^2 + y_i^2} - r,$$

with

$$\frac{dF_i}{dr} = -1, \quad \frac{d^2F_i}{dr^2} = 0.$$

Therefore, the gradient and Hessian of f_{ex1} are

$$\begin{aligned} g(r) &= \nabla f_{\text{ex1}}(r) = \sum_{i=1}^m F_i(r) \frac{dF_i}{dr} = - \sum_{i=1}^m F_i(r) = - \sum_{i=1}^m \left(\sqrt{x_i^2 + y_i^2} - r \right) \\ &= m r - \sum_{i=1}^m \sqrt{x_i^2 + y_i^2}, \end{aligned}$$

and

$$g'(r) = H_{f_{\text{ex1}}}(r) = \sum_{i=1}^m \left[\left(\frac{dF_i}{dr} \right)^2 + F_i(r) \frac{d^2F_i}{dr^2} \right] = \sum_{i=1}^m 1 = m,$$

where m denotes the number of data points. Consequently, the Newton step becomes particularly simple:

$$r_{k+1} = r_k - \frac{g(r_k)}{g'(r_k)} = r_k + \frac{1}{m} \sum_{i=1}^m F_i(r_k) = \frac{1}{m} \sum_{i=1}^m \sqrt{x_i^2 + y_i^2}.$$

Thus, in a single iteration the method already yields the exact solution, namely the average distance of the data points from the origin. $\square$

Example 2.4.3 (Application to f_{ex2}) We now consider the case

$$f_{\text{ex2}}(r, c_x) := f(r, c_x, 0),$$

where the circle center lies on the x -axis. The y -coordinate c_y is fixed to zero, while r and c_x are unknown. The residuals are

$$F_i(r, c_x) = \sqrt{(x_i - c_x)^2 + y_i^2} - r.$$

The partial derivatives of F_i are

$$\frac{\partial F_i}{\partial r} = -1, \quad \frac{\partial F_i}{\partial c_x} = \frac{c_x - x_i}{\sqrt{(x_i - c_x)^2 + y_i^2}},$$

and the nonzero second derivatives are

$$\frac{\partial^2 F_i}{\partial c_x^2} = \frac{y_i^2}{\left((x_i - c_x)^2 + y_i^2 \right)^{3/2}}.$$

Away from these points, F_i is twice continuously differentiable, and the only nonzero second partial derivative is

$$\frac{\partial^2 F_i}{\partial c_x^2}(r, c_x) = \frac{y_i^2}{\left((x_i - c_x)^2 + y_i^2\right)^{3/2}}.$$

With $x = (r, c_x)^T$, the gradient and Hessian of f_{ex2} follow from

$$g(x) = \nabla f_{\text{ex2}}(x) = \sum_{i=1}^m F_i(x) \nabla F_i(x), \quad g'(x) = \sum_{i=1}^m \left(\nabla F_i(x) \nabla F_i(x)^T + F_i(x) \nabla^2 F_i(x) \right).$$

Explicitly,

$$g(x) = \sum_{i=1}^m \begin{pmatrix} -F_i(x) \\ F_i(x) \frac{c_x - x_i}{\sqrt{(x_i - c_x)^2 + y_i^2}} \end{pmatrix},$$

and

$$g'(x) = \sum_{i=1}^m \begin{pmatrix} 1 & -\frac{c_x - x_i}{\sqrt{(x_i - c_x)^2 + y_i^2}} \\ -\frac{c_x - x_i}{\sqrt{(x_i - c_x)^2 + y_i^2}} & \frac{(c_x - x_i)^2}{(x_i - c_x)^2 + y_i^2} + F_i(x) \frac{y_i^2}{\left((x_i - c_x)^2 + y_i^2\right)^{3/2}} \end{pmatrix}.$$

In this two-dimensional setting, the Newton update

$$x^{k+1} = x^k - g'(x^k)^{-1} g(x^k)$$

can be computed iteratively. For noise-free data and a starting value close to the solution, the method converges quadratically to the true parameters (r, c_x) . $\square$

Example 2.4.4 (Application to f_{ex3})

We now consider the general case

$$f_{\text{ex3}}(r, c_x, c_y) := f(r, c_x, c_y),$$

where all three parameters are unknown. The residuals are

$$F_i(r, c_x, c_y) = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2} - r.$$

For brevity, we define

$$d_x := c_x - x_i, \quad d_y := c_y - y_i, \quad \rho := \sqrt{d_x^2 + d_y^2}.$$

Then the first derivatives of F_i are

$$\frac{\partial F_i}{\partial r} = -1, \quad \frac{\partial F_i}{\partial c_x} = \frac{d_x}{\rho}, \quad \frac{\partial F_i}{\partial c_y} = \frac{d_y}{\rho}.$$

Hence, F_i is continuously differentiable everywhere except at the isolated points where $\rho = 0$, i.e. $(c_x, c_y) = (x_i, y_i)$. Consequently, f_{ex3} is continuous on $\mathbb{R}^3$ but not differentiable at these points.

Away from these points, F_i is twice continuously differentiable, and the only nonzero second partial derivatives are

$$\frac{\partial^2 F_i}{\partial c_x^2} = \frac{d_y^2}{\rho^3}, \quad \frac{\partial^2 F_i}{\partial c_y^2} = \frac{d_x^2}{\rho^3}, \quad \frac{\partial^2 F_i}{\partial c_x \partial c_y} = -\frac{d_x d_y}{\rho^3}.$$

With $x = (r, c_x, c_y)^T$, the gradient and Hessian of f_{ex3} follow from

$$g(x) = \nabla f_{\text{ex3}}(x) = \sum_{i=1}^m F_i(x) \nabla F_i(x), \quad g'(x) = \sum_{i=1}^m \left(\nabla F_i(x) \nabla F_i(x)^T + F_i(x) \nabla^2 F_i(x) \right).$$

Explicitly,

$$g(x) = \sum_{i=1}^m \begin{pmatrix} -F_i(x) \\ F_i(x) \frac{d_x}{\rho} \\ F_i(x) \frac{d_y}{\rho} \end{pmatrix},$$

and

$$g'(x) = \sum_{i=1}^m \begin{pmatrix} 1 & -\frac{d_x}{\rho} & -\frac{d_y}{\rho} \\ -\frac{d_x}{\rho} & \frac{d_x^2}{\rho^2} + F_i(x) \frac{d_y^2}{\rho^3} & \frac{d_x d_y}{\rho^2} - F_i(x) \frac{d_x d_y}{\rho^3} \\ -\frac{d_y}{\rho} & \frac{d_x d_y}{\rho^2} - F_i(x) \frac{d_x d_y}{\rho^3} & \frac{d_y^2}{\rho^2} + F_i(x) \frac{d_x^2}{\rho^3} \end{pmatrix}.$$

In this three-dimensional setting, the Newton update

$$x^{k+1} = x^k - g'(x^k)^{-1} g(x^k)$$

can be computed iteratively. For noise-free data and a starting value close to the solution, the method converges quadratically to the true parameters (r, c_x, c_y) . $\square$

§5 Gauss–Newton

In the previous section (Newton–Raphson method), we have seen that even for comparatively simple examples the computation of $g'(x) = Hf(x)$ can become quite involved. The idea of the Gauss–Newton method is to exploit the specific structure of nonlinear least-squares problems and to simplify the expression for the Hessian.

Recall that for the minimization problem

$$f(x) = \frac{1}{2} \sum_{i=1}^m F_i(x)^2,$$

the gradient and Hessian are given by

$$g(x) = J(x)^T F(x), \quad g'(x) = J(x)^T J(x) + \sum_{\ell=1}^m F_\ell(x) H F_\ell(x),$$

where $J(x)$ denotes the Jacobian matrix of $F(x)$,

$$J(x) = \begin{pmatrix} (\nabla F_1(x))^T \\ \vdots \\ (\nabla F_m(x))^T \end{pmatrix}.$$

At the exact solution $\hat{x}$, the residuals vanish for noise-free data, that is,

$$F_i(\hat{x}) = 0, \quad i = 1, \dots, m.$$

If the data are only slightly perturbed, we may still assume that $F_i(x) \approx 0$ for x sufficiently close to $\hat{x}$. Under this assumption, the second term in the Hessian expression becomes small, and we obtain the approximation

$$g'(x) = J(x)^T J(x) + \sum_{\ell=1}^m F_\ell(x) H_{F_\ell}(x) \approx J(x)^T J(x).$$

This motivates the *Gauss–Newton method*, in which the full Hessian $Hf(x)$ is replaced by the simpler approximation $J(x)^T J(x)$. The resulting linear system for the Newton step is

$$J(x_k)^T J(x_k) s_k = -J(x_k)^T F(x_k),$$

and the iterate is updated according to

$$x_{k+1} = x_k + s_k.$$

The complete algorithm is summarized in [Figure 2.3](#), which also shows the corresponding Matlab implementation used in the examples below.

Although we postpone a detailed convergence analysis, it is instructive to note that the Gauss–Newton iteration can be viewed as a fixed point iteration

$$x_{k+1} = \Phi(x_k),$$

where

$$\Phi(x) = x - \left(J(x)^T J(x) \right)^{-1} J(x)^T F(x).$$

For this iteration to be well-defined, it is necessary that

$$\text{rank}(J(x)) = n,$$

so that the matrix $J(x)^T J(x)$ is invertible.

Local convergence can then, in principle, be established using Banach’s fixed point theorem, provided that in a neighbourhood of the solution $\hat{x}$ the mapping Φ is self-mapping and

$$\|\Phi'(\hat{x})\| < 1,$$

where $\|\cdot\|$ denotes any matrix norm on $\mathbb{R}^n$.

Figure 2.3: *Gauss–Newton method and corresponding Matlab implementation.*

Algorithm 1: Gauss–Newton algorithm

Data: Initial guess x_0 , maximum iteration count $n_{\max}$

Result: Approximate solution x_k

```

1 for  $k = 0, 1, 2, \dots, n_{\max}$  do
2   Compute residuals  $b = F(x_k)$  and Jacobian  $A = J(x_k)$ ;
3   Solve  $(A^T A)s_k = -A^T b$ ;
4   Update  $x_{k+1} = x_k + s_k$ ;
5   if  $\|b\| \leq 10^{-9}$  then
6     break;

```

```

1 function [x, k] = gauss_newton(x, F, F_, n_max)
2   arguments
3     x double
4     F (1,1) function_handle
5     F_ (1,1) function_handle
6     n_max (1,1) double {mustBeInteger, mustBeNonnegative}
7   end
8
9   for k = 1:n_max
10    b = F(x);
11    A = F_(x);
12    s = (A'*A) \ (-A'*b);
13    x = x + s;
14    if norm(b) <= 1e-9
15      break;
16    end
17  end
18 end

```

§6 Outlook: Levenberg–Marquardt method

Similar to the Gauss–Newton approach, the idea of the Levenberg–Marquardt method is to simplify the Hessian $H_f(x)$, yet in a way that yields a more robust approximation than Gauss–Newton does. The method replaces the Hessian by a regularized form

$$g'(x) \approx J(x)^T J(x) + \lambda I, \quad \lambda \geq 0,$$

where λ is a damping (or regularization) parameter and I denotes the identity matrix.

In the next chapter, which directly follows, we will introduce and analyze the *gradient-descent method* and *trust region methods*. Both of these concepts play an important role here: the gradient-descent method represents the limiting case of Levenberg–Marquardt for large λ , while trust region methods provide a systematic framework for choosing an appropriate damping parameter λ .

The parameter λ thus controls the transition between the Gauss–Newton and a gradient-descent-like behaviour: for $\lambda = 0$ the method coincides with Gauss–Newton, while for large λ the step direction approaches that of steepest descent.

Choosing a “good” value for λ and understanding why this modification can lead to an improved approximation will be discussed later in the context of trust region methods. Nevertheless, the Levenberg–Marquardt method is already introduced and used in the exercises in order to gain numerical experience with this practically important approach.

Chapter 3 Unconstrained optimization

In this chapter we consider general optimization problems of the form

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x),$$

where the objective function $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ is defined on an open domain Ω . Such problems are called *unconstrained*, because all points $x \in \mathbb{R}^n$ are admissible; there are no explicit equality or inequality constraints restricting the feasible set.

As the following examples illustrate, least squares problems are special cases of unconstrained optimization problems, since the objective function possesses an additional structural form. Consequently, methods, optimality conditions, and convergence results that are derived in this chapter can be directly applied to least squares problems.

Example 3.0.1

- (a) For $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ twice continuously differentiable and

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = \frac{1}{2} \|F(x)\|_2^2,$$

we recover the least squares problem of [Definition 2.1.1](#).

- (b) The Rosenbrock function

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f(x) = f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2,$$

defines a nonlinear optimization problem with a unique global minimum $\hat{x} = (1, 1)^T$, where $f(\hat{x}) = 0$. It is often used as a benchmark to test numerical optimization methods, since it exhibits steep gradients in one direction and very flat ones in another. Moreover, these directions are not aligned with the coordinate axes, resulting in a characteristic “banana-shaped” valley (see [Figure 3.1](#)).

- (c) The Himmelblau function

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2,$$

is a polynomial of degree four and serves as another popular benchmark problem for testing optimization algorithms. It possesses four global minima, each with value 0, approximately at

$$(3.0, 2.0), (-2.805, -3.131), (-3.779, -3.283), (3.584, -1.848).$$

In addition, the function has one local maximum at approximately $(-0.270, -0.924)$ and two saddle points near $(-0.270, -0.924)$ and $(-0.550, 1.830)$, where the gradient vanishes but the Hessian is indefinite. The characteristic shape of the function is illustrated in [Figure 3.2](#).

□

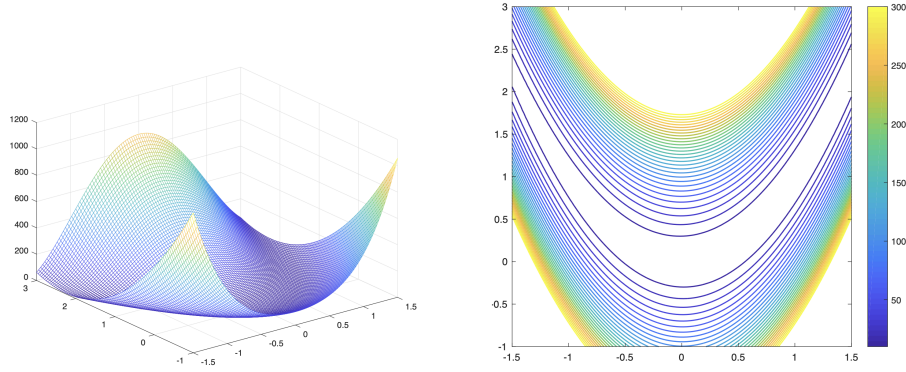


Figure 3.1: Contour plot of the Rosenbrock function $f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$. The narrow, curved valley illustrates the difficulty of optimization methods based solely on local gradient information.

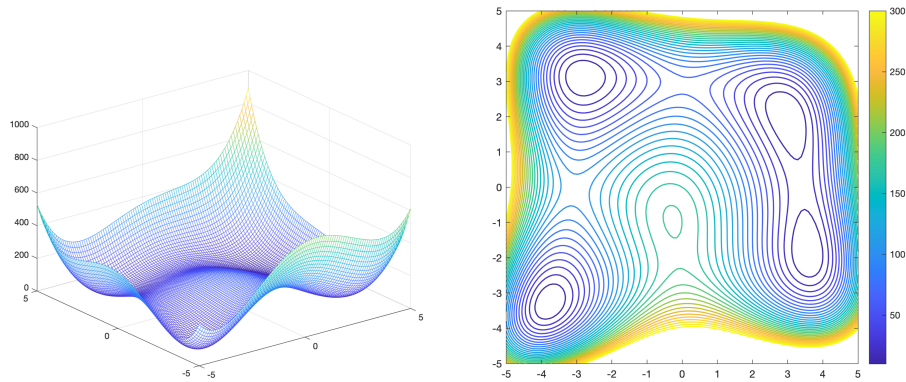


Figure 3.2: Contour plot of the Himmelblau function $f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$. The function has four global minima ($f = 0$), one local maximum, and two saddle points, making it a popular benchmark for nonconvex optimization.

§1 Optimality conditions and convexity

In this section we recall several necessary and sufficient conditions for local and global extrema. While these results are well known from basic analysis courses, they are summarized here in a form suitable for the context of unconstrained optimization.

Definition 3.1.1 (Local extrema)

Let $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ with Ω open.

- (a) f has a *local minimum* at $\hat{x} \in \Omega$ if there exists $\delta > 0$ such that

$$f(x) \geq f(\hat{x}) \quad \forall x \in U_\delta(\hat{x}) := \{x \in \mathbb{R}^n : \|x - \hat{x}\|_2 < \delta\}.$$

In this case, $\hat{x}$ is called a *local minimizer* of f (sometimes also called a *local minimum point*).

If the inequality is strict for all $x \neq \hat{x}$, i.e.

$$f(x) > f(\hat{x}) \quad \forall x \in \dot{U}_\delta(\hat{x}) := \{x : 0 < \|x - \hat{x}\|_2 < \delta\},$$

then f is said to have a *strict local minimum* at $\hat{x}$, and $\hat{x}$ is called a *strict local minimizer* (sometimes also called a *local minimum point*).

(b) f has a *local maximum* at $\hat{x} \in \Omega$ if there exists $\delta > 0$ such that

$$f(x) \leq f(\hat{x}) \quad \forall x \in U_\delta(\hat{x}),$$

In this case, $\hat{x}$ is called a *local maximizer* of f (sometimes also called a *local maximum point*).

If the inequality is strict for all $x \neq \hat{x}$, i.e.

$$f(x) < f(\hat{x}) \quad \forall x \in \dot{U}_\delta(\hat{x}) := \{x : 0 < \|x - \hat{x}\|_2 < \delta\},$$

then f is said to have a *strict local maximum* at $\hat{x}$, and $\hat{x}$ is called a *strict local maximizer* (sometimes also called a *local maximum point*).

□

Theorem 3.1.2 (First-order necessary optimality condition)

Let $f \in C^1(M; \mathbb{R})$, where $M \subseteq \mathbb{R}^n$ is open. If f attains a local minimum at $\hat{x} \in M$, then

$$\nabla f(\hat{x}) = \mathbf{0}.$$

Proof: Let $\delta > 0$ be such that

$$f(x) \geq f(\hat{x}) \quad \forall x \in A := U_\delta(\hat{x}) \subseteq M.$$

For an arbitrary direction $d \in \mathbb{R}^n \setminus \{0\}$ with $\|d\|_2 = 1$, define the curve

$$h_d : (-\delta, \delta) \rightarrow A, \quad h_d(t) = \hat{x} + td,$$

and the scalar function

$$g_d : (-\delta, \delta) \rightarrow \mathbb{R}, \quad g_d(t) = f(h_d(t)) = f(\hat{x} + td).$$

Then

$$g_d(0) = f(\hat{x}), \quad g_d(t) = f(\hat{x} + td) \geq f(\hat{x}) = g_d(0) \quad \forall t \in (-\delta, \delta).$$

Hence,

$$\lim_{t \rightarrow 0^+} \frac{g_d(t) - g_d(0)}{t} \geq 0, \quad \lim_{t \rightarrow 0^-} \frac{g_d(t) - g_d(0)}{t} \leq 0.$$

Since both one-sided derivatives exist and g_d is differentiable, it follows that

$$g'_d(0) = 0. \tag{*}$$

By the chain rule,

$$g'_d(t) = f'(h_d(t))h'_d(t) = \nabla f(h_d(t))^T d.$$

From (*) it follows that

$$\nabla f(\hat{x})^T d = 0 \quad \text{for all directions } d \in \mathbb{R}^n \text{ with } \|d\|_2 = 1.$$

Choosing the canonical basis vectors e_i for $i = 1, \dots, n$, we obtain

$$0 = \nabla f(\hat{x})^T e_i = \frac{\partial f}{\partial x_i}(\hat{x}), \quad i = 1, \dots, n.$$

Hence, all partial derivatives vanish at $\hat{x}$, and therefore

$$\nabla f(\hat{x}) = \mathbf{0}.$$

□

The necessary (but not sufficient) condition for local extrema motivates the following definition.

Definition 3.1.3 (Stationary points and saddle points)

Let $f \in C^1(M; \mathbb{R})$, where $M \subseteq \mathbb{R}^n$ is open.

- (a) A point $\hat{x} \in M$ is called a *stationary point* if

$$\nabla f(\hat{x}) = 0.$$

- (b) A stationary point $\hat{x} \in M$ is called a *saddle point* if it is neither a local minimum nor a local maximum; that is, if for every $\delta > 0$ there exist points $a, b \in U_\delta(\hat{x}) \cap M$ such that

$$f(a) < f(\hat{x}) \quad \text{and} \quad f(b) > f(\hat{x}).$$

□

Remark 3.1.4 (Stability of positive definiteness of the Hessian)

For the forthcoming sufficient optimality condition we will assume that $f \in C^2(\Omega)$ on an open set $\Omega \subseteq \mathbb{R}^n$.

If the Hessian $Hf(\hat{x})$ is positive definite at $\hat{x} \in \Omega$, then $Hf(x)$ is also positive definite on some open neighbourhood $U_\delta(\hat{x}) \subseteq \Omega$. In this remark we write

$A \succ 0$ to denote that A is positive definite.

Proof sketch (via Sylvester's criterion).

By Sylvester's criterion, $Hf(\hat{x}) \succ 0$ iff all leading principal minors

$$D_k(\hat{x}) := \det\left((Hf(\hat{x}))_{1:k, 1:k}\right), \quad k = 1, \dots, n,$$

are strictly positive. Since $f \in C^2$, every entry of $Hf(x)$ is continuous, hence each $D_k(x)$ is continuous. Therefore, for each k there exists $\delta_k > 0$ such that

$$D_k(x) > \frac{1}{2}D_k(\hat{x}) > 0 \quad \forall x \in U_{\delta_k}(\hat{x}).$$

With $\delta := \min_k \delta_k$ we obtain $D_k(x) > 0$ for all k and all $x \in U_\delta(\hat{x})$. Hence $Hf(x) \succ 0$ there. □

Theorem 3.1.5 (Second-order sufficient optimality condition)

Let $f \in C^2(M; \mathbb{R})$, where $M \subseteq \mathbb{R}^n$ is open, and let $\hat{x} \in M$ satisfy:

- (i) $\nabla f(\hat{x}) = 0$,
(ii) $Hf(\hat{x})$ is positive definite; that is,

$$d^T Hf(\hat{x}) d > 0 \quad \forall d \in \mathbb{R}^n \setminus \{0\}.$$

Then $\hat{x}$ is a strict local minimizer of f .

Proof: By continuity of the Hessian Hf , there exists $\delta > 0$ such that $Hf(x) \succ 0$ for all $x \in U_\delta(\hat{x})$. For arbitrary $x \in \dot{U}_\delta(\hat{x})$, define the path

$$h : [0, 1] \rightarrow \mathbb{R}^n, \quad h(t) = \hat{x} + td, \quad d := x - \hat{x},$$

and the scalar function

$$g : [0, 1] \rightarrow \mathbb{R}, \quad g(t) = f(h(t)).$$

Then $g(0) = f(\hat{x})$ and $g(1) = f(x)$. By Taylor's theorem, there exists $\xi \in (0, 1)$ such that

$$g(1) = g(0) + g'(0) + \frac{1}{2}g''(\xi).$$

By the chain rule,

$$g'(t) = \nabla f(h(t))^T h'(t) = \nabla f(h(t))^T d \quad \Rightarrow \quad g'(0) = \nabla f(\hat{x})^T d = 0,$$

by assumption (i). Differentiating again,

$$g''(t) = d^T Hf(h(t)) d.$$

Since $Hf(h(t))$ is positive definite for all $t \in [0, 1]$, we have $g''(t) > 0$, and therefore

$$g(1) - g(0) = \frac{1}{2}g''(\xi) = \frac{1}{2}d^T Hf(h(\xi)) d > 0.$$

Hence $f(x) = g(1) > g(0) = f(\hat{x})$. Since $x \in \dot{U}_\delta(\hat{x})$ was arbitrary, $\hat{x}$ is a strict local minimizer of f . $\square$

Theorem 3.1.6 (Second-order necessary optimality condition (minimum))

Let $f \in C^2(\Omega)$ on an open set $\Omega \subseteq \mathbb{R}^n$ and let $\hat{x} \in \Omega$ be a local minimizer of f . Then

$$\nabla f(\hat{x}) = 0 \quad \text{and} \quad Hf(\hat{x}) \succeq 0.$$

Sketch / exercise: The first part follows from the first-order necessary condition. To show that the Hessian is positive semidefinite, argue analogously to the proof of the sufficient condition, using that H_f is continuous and that f attains a local minimum at $\hat{x}$.

Definition 3.1.7 (Convex sets)

Let $M \subseteq \mathbb{R}^n$.

- (a) M is *convex* if for any two points $x_1, x_2 \in M$ and any every $\lambda \in [0, 1]$, the convex combination, i.e. $\lambda x_1 + (1 - \lambda)x_2$, also belongs to M , that is

$$x_1, x_2 \in M \quad \Rightarrow \quad \lambda x_1 + (1 - \lambda)x_2 \in M \quad \forall \lambda \in [0, 1].$$

- (b) M is *strictly convex* if the above holds with strict inclusion for all $x_1 \neq x_2$, i.e.

$$x_1, x_2 \in M, \quad x_1 \neq x_2 \quad \Rightarrow \quad \lambda x_1 + (1 - \lambda)x_2 \in \mathring{M} \quad \forall \lambda \in (0, 1),$$

where $\mathring{M} := \{x \in M : \exists \varepsilon > 0 : U_\varepsilon(x) \subseteq M\}$.

- (c) The *convex hull* of a non-empty M , denoted $\text{conv}(M)$, is the smallest convex set containing M , that is

$$\text{conv}(M) := \bigcap \{C \subseteq \mathbb{R}^n : C \text{ convex}, M \subseteq C\}.$$

It can be shown that $\text{conv}(M)$ can equivalently be expressed as the set of all convex combinations of finite many points of M :

$$\text{conv}(M) = \left\{ \sum_{i=1}^k \lambda_i x_i \mid x_i \in M, \lambda_i \geq 0, \sum_{i=1}^k \lambda_i = 1, k \in \mathbb{N} \right\}.$$

□

Definition 3.1.8 (Convex functions)

Let $M \subseteq \mathbb{R}^n$ be convex and $f : M \rightarrow \mathbb{R}$.

- (a) Function f is said to be *convex* if

$$f((1 - \lambda)x + \lambda y) \leq (1 - \lambda)f(x) + \lambda f(y) \quad \forall x, y \in M, \lambda \in [0, 1].$$

- (b) Function f is said to be *strictly convex* if

$$f((1 - \lambda)x + \lambda y) < (1 - \lambda)f(x) + \lambda f(y) \quad \forall x, y \in M, x \neq y, \lambda \in (0, 1).$$

- (c) Function f is said to be *uniformly convex* if there exists $\mu > 0$ such that

$$\lambda f(x) + (1 - \lambda)f(y) - f(\lambda x + (1 - \lambda)y) \geq \mu \lambda(1 - \lambda)\|x - y\|_2^2$$

for all $x, y \in M$ and $\lambda \in [0, 1]$.

□

Theorem 3.1.9 (First-order characterizations of convexity)

Let $M \subseteq \mathbb{R}^n$ be convex and $f \in C^1(M; \mathbb{R})$.

- (a) f is convex if and only if

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) \quad \forall x, y \in M. \quad (3.1)$$

- (b) f is strictly convex if and only if

$$f(y) > f(x) + \nabla f(x)^T(y - x) \quad \forall x, y \in M, x \neq y.$$

- (c) f is uniformly (strongly) convex if and only if there exist $\mu > 0$ such that

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) + \mu\|y - x\|_2^2 \quad \forall x, y \in M.$$

Proof:

(a) “ $\Rightarrow$ ”: Let $x, y \in M$ be arbitrary but fixed. From the convexity of f it follows that

$$f(x + t(y - x)) = f((1 - t)x + ty) \leq (1 - t)f(x) + tf(y) = f(x) + t(f(y) - f(x))$$

for all $t \in [0, 1]$. Hence,

$$\frac{f(x + t(y - x)) - f(x)}{t} \leq f(y) - f(x) \quad \forall t \in [0, 1]. \quad (3.2)$$

Consider now

$$h : [0, 1] \rightarrow M, \quad h(t) = x + t(y - x), \quad g : [0, 1] \rightarrow \mathbb{R}, \quad g(t) = f(h(t)).$$

Then g is continuously differentiable, and by the chain rule,

$$g'(0) = f'(h(0))h'(0) = \nabla f(x)^T(y - x).$$

From (3.2) we obtain

$$g'(0) = \lim_{t \rightarrow 0^+} \frac{f(x + t(y - x)) - f(x)}{t} \leq f(y) - f(x),$$

and therefore

$$f(y) \geq f(x) + \nabla f(x)^T(y - x).$$

“ $\Leftarrow$ ”: Assume that (3.1) holds, and let

$$x_\lambda := \lambda x + (1 - \lambda)y, \quad \lambda \in [0, 1],$$

be arbitrary but fixed. Then

$$f(x) \geq f(x_\lambda) + \nabla f(x_\lambda)^T(x - x_\lambda) = f(x_\lambda) - \nabla f(x_\lambda)^T x_\lambda + \nabla f(x_\lambda)^T x,$$

$$f(y) \geq f(x_\lambda) + \nabla f(x_\lambda)^T(y - x_\lambda) = f(x_\lambda) - \nabla f(x_\lambda)^T x_\lambda + \nabla f(x_\lambda)^T y.$$

Taking the convex combination of these two inequalities yields

$$\begin{aligned} \lambda f(x) + (1 - \lambda)f(y) &\geq f(x_\lambda) - \nabla f(x_\lambda)^T x_\lambda + \lambda \nabla f(x_\lambda)^T x + (1 - \lambda) \nabla f(x_\lambda)^T y \\ &= f(x_\lambda) - \nabla f(x_\lambda)^T x_\lambda + \nabla f(x_\lambda)^T(\lambda x + (1 - \lambda)y) \\ &= f(x_\lambda) - \nabla f(x_\lambda)^T x_\lambda + \nabla f(x_\lambda)^T x_\lambda \\ &= f(x_\lambda) = f(\lambda x + (1 - \lambda)y). \end{aligned}$$

Hence

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y),$$

which shows that f is convex.

(b), (c) Analogous arguments apply.

□

For the sake of completeness, we recall the second-order characterizations of convexity, which follow directly from the first-order conditions and are stated here without proof.

Theorem 3.1.10 (Second-order characterizations of convexity)

Let $M \subseteq \mathbb{R}^n$ be convex and $f \in C^2(M; \mathbb{R})$.

- (a) f is convex if and only if the Hessian $Hf(x)$ is positive semidefinite for all $x \in M$.
- (b) f is strictly convex if the Hessian $Hf(x)$ is positive definite for all $x \in M$.
- (c) f is uniformly (strongly) convex if and only if there exists $\mu > 0$ such that

$$d^T Hf(x) d \geq \mu d^T d \quad \forall x \in M, \forall d \in \mathbb{R}^n.$$

The first- and second-order characterizations of convexity provide the theoretical basis for optimization. Their central implication is that, for convex functions, finding stationary points is sufficient, since each of them corresponds to a local, and therefore also a global, minimum.

Theorem 3.1.11 (Optimality properties of convex functions)

Let $M \subseteq \mathbb{R}^n$ be convex and $f : M \rightarrow \mathbb{R}$ convex.

- (a) Every local minimizer of f is also a global minimizer.
- (b) If f is strictly convex, then f has at most one local minimizer.
- (c) If M is open and $f \in C^1(M; \mathbb{R})$, then every stationary point of f is a (global) minimizer.

Proof:

- (a) Let $\hat{x} \in M$ be a local minimizer of f . Then there exists $\delta > 0$ such that

$$f(\hat{x}) \leq f(z) \quad \forall z \in U_\delta(\hat{x}) \cap M.$$

Assume by contradiction that there is $x \in M$ with $f(x) < f(\hat{x})$ and set

$$\lambda := \frac{\delta}{2\|x - \hat{x}\|_2} \in (0, 1).$$

By construction,

$$\|\hat{x} + \lambda(x - \hat{x}) - \hat{x}\| = \lambda\|x - \hat{x}\|_2 = \frac{\delta}{2} < \delta,$$

hence $x_\lambda := \hat{x} + \lambda(x - \hat{x}) \in U_\delta(\hat{x})$. Since M is convex and $\lambda \in (0, 1)$, we also have $x_\lambda \in M$. By convexity of f ,

$$f(x_\lambda) \leq (1 - \lambda)f(\hat{x}) + \lambda f(x) < (1 - \lambda)f(\hat{x}) + \lambda f(\hat{x}) = f(\hat{x}),$$

which contradicts the local minimality of $\hat{x}$. Therefore, no such x exists and $\hat{x}$ is a global minimizer.

- (b) Let $x_1, x_2 \in M$ be two local minimizers. Then, by part (a),

$$f(x_1) = f(x_2).$$

Since f is strictly convex, we have

$$f(x_1 + \lambda(x_2 - x_1)) < f(x_1) + \lambda(f(x_2) - f(x_1)) = f(x_1) \quad \forall \lambda \in (0, 1).$$

If $x_1 \neq x_2$, define the direction vector

$$d := \frac{x_2 - x_1}{\|x_2 - x_1\|_2}.$$

Then

$$f(x_1 + td) < f(x_1) \quad \forall 0 < t < \|x_2 - x_1\|_2.$$

Hence x_1 cannot be a local minimizer, since for every $\delta > 0$ the point $x = x_1 + \frac{\delta}{2}d$ lies within $U_\delta(x_1)$ and satisfies $f(x) < f(x_1)$. Therefore $x_1 = x_2$, and the minimizer is unique.

(c) Let $\hat{x}$ be a local minimizer of f . Then, by [Theorem 3.1.2](#)

$$\nabla f(\hat{x}) = 0.$$

Using [Theorem 3.1.9](#), we obtain for every $x \in M$ that

$$f(x) \geq f(\hat{x}) + \nabla f(\hat{x})^T(x - \hat{x}) = f(\hat{x}).$$

Hence $\hat{x}$ is also a global minimizer.

□

§2 Direct Search Methods

We now turn to methods that rely solely on evaluations of the objective function at certain points, without requiring derivatives or any further smoothness assumptions. Such schemes are therefore called direct search methods or derivative-free methods. In contrast, methods that make use of derivatives of f (for instance, the gradient) are referred to as gradient-based methods and will be studied later. In general, gradient-based algorithms show better convergence behavior than direct search methods.

Obviously, direct search algorithms are particularly useful for functions that are not differentiable, or for functions whose derivatives are either unknown or computationally expensive to obtain. Moreover, derivative-free methods are often appropriate for oscillating or noisy objective functions (e.g. random functions) where derivatives would be extremely irregular.

In this section, we will not need any of the theoretical considerations discussed so far or those that will follow later in this course. General assertions about the convergence of these methods are not known. Even for seemingly simple two-dimensional problems, these methods do not necessarily converge to a stationary point—and if they do converge, they often do so very slowly.

Nevertheless, in many situations there is simply no alternative: if derivatives cannot be computed, one has to rely on such methods. For this reason, direct search methods are widely used in practice, and every numerical analyst should at least be familiar with their basic ideas.

§2.1 Hooke–Jeeves Method

The algorithm proposed by Hooke and Jeeves (1961) is also known as a pattern search method. It is an iterative scheme that approximates the desired minimizer $x^* \in \mathbb{R}^n$ of f by successively searching along the coordinate axes.

Starting from an initial approximation $x^{(0)} \in \mathbb{R}^n$ of x^* , the next (and hopefully improved) approximation $x^{(k+1)}$ is computed in two steps:

1. *the exploration step.*
2. *the progression step.*

We describe these steps in more detail below.

Exploration Step. In the exploration step, one explores the orthogonal coordinate directions starting from $x^{(k)}$ in order to identify in which direction the objective function f decreases most.

Let $e_1, \dots, e_n \in \mathbb{R}^n$ denote the canonical basis vectors, $h = (h_1, \dots, h_n)^T \in \mathbb{R}_+^n$ the vector of step sizes with $h_j > 0$, and $x^{(k)}$ the current iterate. Then, for each coordinate direction, we check whether

$$f(x^{(k)} + h_j e_j) < f(x^{(k)}), \quad j = 1, \dots, n.$$

If this holds for an index j , we update $x^{(k)} \leftarrow x^{(k)} + h_j e_j$.

Otherwise, if $f(x^{(k)} + h_j e_j) \geq f(x^{(k)})$, we test in the opposite direction:

$$f(x^{(k)} - h_j e_j) < f(x^{(k)}), \quad j = 1, \dots, n$$

and if successful an index j , we update $x^{(k)} \leftarrow x^{(k)} - h_j e_j$.

After testing all n directions (requiring at most $2n + 1$ evaluations), we obtain some vector $y^{(k)}$ satisfying $f(y^{(k)}) \leq f(x^{(k)})$. Two cases can occur:

- (i) *No improvement:* If $y^{(k)} = x^{(k)}$, no progress was made. The step size is then halved, $h \leftarrow \frac{1}{2}h$, and a new exploration step is started from $x^{(k)}$.
- (ii) *Improvement:* If $y^{(k)} \neq x^{(k)}$, we found a direction of decrease. A progression step is performed.

Progression Step. In the progression step, a line search is carried out along the line through $x^{(k)}$ and $y^{(k)}$, in particular beyond $y^{(k)}$. A typical choice is $t = 1$, leading to

$$w^{(k)} := 2y^{(k)} - x^{(k)} = x^{(k)} + 2(y^{(k)} - x^{(k)}).$$

Starting from $w^{(k)}$, another exploration step with the current step size is performed. If this yields a further improvement, i.e. some $z^{(k)}$ with $f(z^{(k)}) < f(y^{(k)})$, we set $x^{(k+1)} := z^{(k)}$; otherwise, $x^{(k+1)} := y^{(k)}$.

Stopping Criterion. A common stopping criterion is

$$\|x^{(k+1)} - x^{(k)}\|_1 \leq \Delta,$$

for a given tolerance $\Delta > 0$.

The complete Hooke–Jeeves method is summarized in [Algorithm 2](#) and [Algorithm 3](#).

Algorithm 2: Hooke–Jeeves method

Data: Objective function $f \in C(\mathbb{R}^n)$, initial vector $x^{(0)} \in \mathbb{R}^n$, initial step sizes $h = (h_1, \dots, h_n)^T \in \mathbb{R}_+^n$, tolerance $\Delta > 0$

Result: Approximation of the minimizer $x^{(k)}$

```

1  $k \leftarrow 0$ ;
2 while  $\|h\|_1 > \Delta$  do
3    $y^{(k)} \leftarrow \text{Exploration}(f, x^{(k)}, h)$ ;
4   if  $y^{(k)} = x^{(k)}$  then
5      $h \leftarrow \frac{1}{2}h$ ;
6   else
7      $w^{(k)} \leftarrow 2y^{(k)} - x^{(k)}$ ;
8      $z^{(k)} \leftarrow \text{Exploration}(f, w^{(k)}, h)$ ;
9     if  $f(z^{(k)}) < f(y^{(k)})$  then
10       $x^{(k+1)} \leftarrow z^{(k)}$ ;
11     else
12       $x^{(k+1)} \leftarrow y^{(k)}$ ;
13    $k \leftarrow k + 1$ ;

```

Algorithm 3: Exploration procedure

Data: $f \in C(\mathbb{R}^n)$, current point $x \in \mathbb{R}^n$, step sizes $h \in \mathbb{R}_+^n$

Result: Updated point x after exploration

```

1 for  $j = 1, \dots, n$  do
2   if  $f(x + h_j e_j) < f(x)$  then
3      $x \leftarrow x + h_j e_j$ ;
4   else
5     if  $f(x - h_j e_j) < f(x)$  then
6        $x \leftarrow x - h_j e_j$ ;
7 return  $x$ ;

```

Remark 3.2.1

The initial step sizes $h_j > 0$ should be chosen such that the quantities $|f(x^{(k)} \pm h_j e_j) - f(x^{(k)})|$, for $j = 1, \dots, n$, have roughly the same magnitude, in order to ensure balanced accuracy. Since the shape of f is typically unknown, this choice can be quite delicate in practice. $\square$

§2.2 Nelder–Mead Method

The derivative-free minimization scheme introduced by Nelder and Mead (1965) is widely used in practice (e.g., Matlab’s `fminsearch`).

A *polytope* is the convex hull of a finite number of vectors $x_1, \dots, x_m \in \mathbb{R}^n$. If these vectors are *affinely independent*,¹ we call the polytope a *simplex* with the vertices $x_1, \dots, x_m$.

However, in this subsection we will use the term “polytope” as a synonym for “simplex,” always assuming the affine independence of the generating points.

Suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a given function, and that we have an n -dimensional polytope with $n + 1$ vertices $x_1, x_2, \dots, x_{n+1}$. Let these be ordered such that

$$f_1 \leq f_2 \leq \dots \leq f_{n+1}, \quad f_j := f(x_j).$$

In each iteration step, the currently worst vertex x_{n+1} is replaced by a new vertex. We denote by

$$x^c := \frac{1}{n} \sum_{j=1}^n x_j$$

the barycenter (or centroid) of the best n vertices. The polytope is then modified using the operations

Reflection – Expansion – Contraction – Shrinking.

At the beginning of each iteration we attempt a reflection of the polytope, computed as

$$x^r := x^c + \alpha(x^c - x_{n+1}), \quad f^r := f(x^r),$$

where $\alpha > 0$ is a fixed constant (commonly $\alpha = 1$). The parameter α is called the reflection coefficient, and x^r the reflection point.

Then we consider the following three cases:

- (1) If $f_1 \leq f^r \leq f_n$, we simply replace x_{n+1} with the reflection point x^r .
- (2) If $f^r < f_1$, the direction of the reflection appears promising for further exploration. Therefore, we expand the polytope and compute

$$x^e := x^c + \beta(x^r - x^c), \quad f^e := f(x^e),$$

with a fixed constant $\beta > 1$ (often $\beta = 2$). The parameter β is called the expansion coefficient, and x^e the extrapolation point.

If $f^e < f^r$, $x_{n+1} \leftarrow x^e$; else $x_{n+1} \leftarrow x^r$.

- (3) If $f^r \geq f_n$, the polytope appears to be too large. In this case we attempt a (partial) contraction of the polytope.

- (a) If $f^r \geq f_{n+1}$, we compute

$$x^k := x^c + \gamma(x_{n+1} - x^c), \quad f^k := f(x^k),$$

with a fixed constant $\gamma \in (0, 1)$ (commonly $\gamma = \frac{1}{2}$). The parameter γ is called the contraction coefficient, and x^k the contraction point. If $f^k < f_{n+1}$, we replace x_{n+1} with x^k ; otherwise, we shrink the entire polytope according to

$$\hat{x}_j := \sigma(x_j + x_1), \quad j = 2, \dots, n + 1.$$

with a fixed constant $\sigma \in (0, 1)$ (commonly $\sigma = \frac{1}{2}$).

¹Vectors $x_1, \dots, x_m \in \mathbb{R}^n$ are called affinely independent if the differences $x_2 - x_1, \dots, x_m - x_1$ are linearly independent. Equivalently, no point of the set can be written as an affine combination of the others.

(b) If $f^r < f_{n+1}$, we compute

$$x^k := x^c + \gamma(x^r - x^c).$$

If $f^k \leq f^r$, we replace x_{n+1} with x^k ; otherwise, we shrink the polytope as in described in (a).

After each iteration, the vertices are re-labeled such that

$$f_1 \leq f_2 \leq \cdots \leq f_{n+1}$$

holds again.

Termination condition. The iteration terminates when

$$\frac{1}{n+1} \sum_{j=1}^{n+1} |f(x_j) - f(x^c)|^2 < \varepsilon^2,$$

for a prescribed tolerance $\varepsilon > 0$. This condition indicates that the function values at the vertices are nearly constant, i.e. the polytope is located in a region where f varies only slightly.

Algorithm 4: Nelder–Mead method (in the notation of the subsection)

Data: Objective $f : \mathbb{R}^n \rightarrow \mathbb{R}$; initial simplex $S^{(0)} = \{x_1, \dots, x_{n+1}\}$; parameters $\alpha > 0$ (reflection), $\gamma > 1$ (expansion), $\beta \in (0, 1)$ (contraction), $\sigma \in (0, 1)$ (shrink); tolerance $\varepsilon > 0$; optional max. iterations $K_{\max}$

Result: Approximate minimizer and final simplex

```
1  $k \leftarrow 0$ ;  
2 repeat  
3   Relabel  $\{x_1, \dots, x_{n+1}\}$  so that  $f_1 \leq f_2 \leq \dots \leq f_{n+1}$ ;  
4    $x^c \leftarrow \frac{1}{n} \sum_{j=1}^n x_j$ ;  
5    $x^r \leftarrow x^c + \alpha(x^c - x_{n+1})$ ,  $f^r \leftarrow f(x^r)$ ;  
6   if  $f_1 \leq f^r \leq f_n$  then  
7      $x_{n+1} \leftarrow x^r$ ;  
8   else if  $f^r < f_1$  then  
9      $x^e \leftarrow x^c + \beta(x^r - x^c)$ ,  $f^e \leftarrow f(x^e)$ ;  
10    if  $f^e < f^r$  then  
11       $x_{n+1} \leftarrow x^e$ ;  
12    else  
13       $x_{n+1} \leftarrow x^r$ ;  
14  else if  $f^r \geq f_n$  then  
15    if  $f^r \geq f_{n+1}$  then  
16       $x^k \leftarrow x^c + \gamma(x_{n+1} - x^c)$ ,  $f^k \leftarrow f(x^k)$ ;  
17      if  $f^k < f_{n+1}$  then  
18         $x_{n+1} \leftarrow x^k$ ;  
19      else  
20        for  $j = 2, \dots, n+1$  do  
21           $x_j \leftarrow x_1 + \sigma(x_j - x_1)$ ;  
22      else  
23         $x^k \leftarrow x^c + \gamma(x^r - x^c)$ ,  $f^k \leftarrow f(x^k)$ ;  
24        if  $f^k \leq f^r$  then  
25           $x_{n+1} \leftarrow x^k$ ;  
26        else  
27          for  $j = 2, \dots, n+1$  do  
28             $x_j \leftarrow x_1 + \sigma(x_j - x_1)$ ;  
29     $f_1 \leq f_2 \leq \dots \leq f_{n+1}$ ;  
30     $k \leftarrow k + 1$ ;  
31 until  $\frac{1}{n+1} \sum_{j=1}^{n+1} |f(x_j) - f(x^c)|^2 < \varepsilon^2$  or  $k > K_{\max}$ ;
```

§2.3 Numerical Experiments: Rosenbrock Function

In the lecture, we will demonstrate the behavior of the Hooke–Jeeves and Nelder–Mead methods using the Rosenbrock function as a test problem. In the exercises, you will implement these methods yourself and compare their performance.

§3 Descent Methods with Line Search

In this class of methods, we assume that the objective function f is continuously differentiable. Descent methods with line search form a broad family of algorithms that generate a sequence

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots \in \mathbb{R}^n.$$

In each iteration, the next iterate $x^{(k+1)}$ is computed from the current iterate $x^{(k)}$ in two steps:

1. A search direction $d^{(k)} \in \mathbb{R}^n \setminus \{0\}$ is determined such that it represents a direction of descent for f at $x^{(k)}$.
2. A step size $\alpha_k \in \mathbb{R}$ is chosen, typically by performing a one-dimensional line search along the direction $d^{(k)}$.

The update rule is then given by

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}. \quad (3.3)$$

Search direction and step size. If the search direction $d^{(k)}$ is chosen such that

$$\nabla f(x^{(k)})^T d^{(k)} < 0, \quad (3.4)$$

in particular implying $d^{(k)} \neq 0$, then there exists a step size $\alpha_k > 0$ such that

$$f(x^{(k+1)}) = f(x^{(k)} + \alpha_k d^{(k)}) < f(x^{(k)}).$$

To see this, consider the Taylor expansion

$$f(x) = f(x^{(k)}) + \nabla f(x^{(k)})^T (x - x^{(k)}) + \xi (x - x^{(k)})^T (x - x^{(k)}), \quad \xi \in (0, 1).$$

Since f is continuously differentiable, and because of (3.4), there exists a $\delta > 0$ such that

$$\nabla f(x)^T d^{(k)} < 0 \quad \forall x \in U_\delta(x^{(k)}).$$

For $0 < \alpha < \frac{\delta}{\|d^{(k)}\|_2} =: \alpha_1$, we have $x^{(k)} + \alpha d^{(k)} \in U_\delta(x^{(k)})$ and consequently

$$f(x^{(k)} + \alpha d^{(k)}) < f(x^{(k)}) \quad \forall \alpha \in (0, \alpha_1).$$

The particular choice $d^{(k)} = -\nabla f(x^{(k)})$ is called the *steepest descent direction*. This terminology is motivated by the fact that

$$-\nabla f(x^{(k)})^T \frac{\nabla f(x^{(k)})}{\|\nabla f(x^{(k)})\|_2} = \min_{d \in \mathbb{R}^n \setminus \{0\}} \nabla f(x^{(k)})^T \frac{d}{\|d\|_2}.$$

These requirements for the search direction, as well as further desirable properties, are summarized in the following section.

§3.1 Basic Concepts

Definition 3.3.1 (Search direction)

A vector $d \in \mathbb{R}^n \setminus \{0\}$ is called a *search direction* for $f \in C^1(\mathbb{R}^n)$ at a point $x \in \mathbb{R}^n$ if

$$\nabla f(x)^T d < 0.$$

□

This concept has already been motivated above. However, we also require additional properties for the sequence of generated search directions, which are summarized in the following definition.

Definition 3.3.2 (Feasible search directions)

Let $(x^{(k)})$ be the sequence of iterates generated by a descent method, and let $(d^{(k)})$ be the corresponding sequence of search directions. The sequence $(d^{(k)})$ is called *feasible* (or *gradient-related*) if the implication

$$\lim_{k \rightarrow \infty} \frac{\nabla f(x^{(k)})^T d^{(k)}}{\|d^{(k)}\|_2} = 0 \implies \lim_{k \rightarrow \infty} \nabla f(x^{(k)}) = 0$$

holds.

□

Intuitively, this means that if the gradient becomes orthogonal to all feasible search directions, the iterates $x^{(k)}$ must approach a stationary point. In other words, the algorithm cannot stagnate away from points where $\nabla f(x) = 0$. This definition therefore characterizes not only particular sequences of search directions $d^{(k)}$, but also the corresponding sequence of iterates $x^{(k)}$.

We have certain degrees of freedom in constructing feasible search directions (and hence the corresponding iterates). In the following, we summarize several important special cases, including the choice of the steepest descent direction.

Remark 3.3.3 (Special cases of search directions)

Important special cases of search directions have the form

$$d^{(k)} = -B_k \nabla f(x^{(k)}), \quad B_k \in \mathbb{R}^{n \times n}.$$

Except for the last case, B_k is symmetric and positive definite.

- (a) $B_k = I$ leads to $d^{(k)} = -\nabla f(x^{(k)})$, the direction of steepest descent discussed in the previous section. This yields the gradient (steepest) descent method.
- (b) $B_k = [Hf(x^{(k)})]^{-1}$ leads to the Newton method.
- (c) $B_k \approx [Hf(x^{(k)})]^{-1}$ gives rise to a quasi-Newton method.
- (d) In the conjugate gradient method, the new search direction is constructed by orthogonalizing (in the sense of conjugacy with respect to the Hessian) against the previous directions, typically using a Gram–Schmidt-like update. Formally, one can still write $d^{(k)} = -B_k \nabla f(x^{(k)})$, but here B_k depends on previous search directions and is not necessarily symmetric or positive definite.

□

§3.2 Line Search and Armijo Rule

A natural idea is to choose α_k such that

$$f(x^{(k)} + \alpha_k d^{(k)}) = \min_{\alpha \in \mathbb{R}} f(x^{(k)} + \alpha d^{(k)}).$$

However, except for special cases (for instance when f is quadratic), this exact line search is usually too expensive to compute, and in general it may not even be possible to determine the minimizer analytically. One such special case was already encountered in the lecture on Numerical Linear Algebra and is briefly recalled here.

Example 3.3.4 (Exact line search for a quadratic objective function)

Let $A \in \mathbb{R}^{n \times n}$ be symmetric and positive definite, $b \in \mathbb{R}^n$, and

$$f(x) = \frac{1}{2}x^T A x - b^T x.$$

For an arbitrary $x_0 \in \mathbb{R}^n$ and direction $d \in \mathbb{R}^n$ we obtain

$$\frac{d}{d\alpha} f(x_0 + \alpha d) = d^T A(x_0 + \alpha d) - b^T d = d^T (Ax_0 - b) + \alpha d^T A d.$$

Setting this derivative to zero yields the stationary point

$$\hat{\alpha} = -\frac{d^T (Ax_0 - b)}{d^T A d}.$$

Furthermore, f is strictly convex since its second derivative

$$\frac{d^2}{d\alpha^2} f(x_0 + \alpha d) = d^T A d$$

is positive for all $d \neq 0$, because A is positive definite. Hence, the stationary point $\hat{\alpha}$ is the unique global minimizer:

$$f(x_0 + \hat{\alpha} d) = \min_{\alpha \in \mathbb{R}} f(x_0 + \alpha d).$$

□

In general, however, the exact line search is neither practical nor necessary. It is usually sufficient to find a step size that yields a sufficient decrease in the objective function. For this purpose, various step size strategies have been developed that require only function and gradient evaluations. In the following, we introduce the Armijo rule, which provides a simple and widely used criterion for choosing the step size when f is merely continuously differentiable. Other line search strategies will be discussed in later sections.

Definition 3.3.5 (Armijo step size)

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuously differentiable, and let $\sigma, \beta \in (0, 1)$ be fixed parameters. For $x \in \mathbb{R}^n$ and a descent direction $d \in \mathbb{R}^n$ satisfying $\nabla f(x)^T d < 0$, the *Armijo step size* is defined as

$$\alpha = \max \left\{ t > 0 : f(x + td) \leq f(x) + \sigma t \nabla f(x)^T d, t = \beta^\ell, \ell \in \mathbb{N}_0 \right\}.$$

□

In the following we will show that under these assumptions there always exists a finite $\ell \in \mathbb{N}_0$ such that $\alpha = \beta^\ell$. Hence, the computation of the Armijo step size is numerically feasible.

Lemma 3.3.6 (Existence of the Armijo step size)

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuously differentiable, and let $\sigma, \beta \in (0, 1)$ be fixed. For $x \in \mathbb{R}^n$ and a descent direction $d \in \mathbb{R}^n$ with $\nabla f(x)^T d < 0$, there exists $\ell^* \in \mathbb{N}_0$ such that

$$f(x + \beta^{\ell^*} d) \leq f(x) + \sigma \beta^{\ell^*} \nabla f(x)^T d.$$

In particular, the set

$$S := \{\ell \in \mathbb{N}_0 : f(x + \beta^\ell d) \leq f(x) + \sigma \beta^\ell \nabla f(x)^T d\}$$

is nonempty. Letting $\ell^* := \min S$, the Armijo step size $\alpha = \beta^{\ell^*}$ is well-defined and can be found by backtracking.

Proof: Set $g := \nabla f(x)$ and write $g^T d = -\gamma$ with $\gamma > 0$. By differentiability of f at x in the direction d , there exists a remainder $r(t)$ such that

$$f(x + td) = f(x) + tg^T d + r(t) \quad \text{with} \quad \lim_{t \rightarrow 0} \frac{r(t)}{t} = 0.$$

Hence, there exists $\delta > 0$ such that

$$|r(t)| \leq \frac{1 - \sigma}{2} \gamma t \quad \forall t \in (0, \delta].$$

For such t we have

$$f(x + td) \leq f(x) + t(-\gamma) + \frac{1 - \sigma}{2} \gamma t = f(x) - \frac{1 + \sigma}{2} \gamma t \leq f(x) - \sigma \gamma t = f(x) + \sigma t g^T d.$$

Thus, the Armijo condition holds for all $t \in (0, \delta]$. Since $\beta^\ell \rightarrow 0$ as $\ell \rightarrow \infty$, there exists $\ell_0 \in \mathbb{N}_0$ with $\beta^{\ell_0} \leq \delta$, and hence the set S is nonempty. Defining $\ell^* := \min S$, we obtain $\alpha = \beta^{\ell^*}$ as the largest candidate step size from $\{\beta^\ell\}$ that satisfies the Armijo condition. $\square$

Remark 3.3.7 (Typical parameter choices) In practice, the parameters of the Armijo rule are usually chosen as

$$\sigma \in [10^{-4}, 10^{-1}], \quad \beta \in [0.1, 0.8].$$

Typical default values are $\sigma = 10^{-4}$ and $\beta = 0.5$. The parameter σ controls how much decrease in f is required (small values allow larger step sizes), while β determines how strongly the step size is reduced during backtracking. $\square$

§3.3 Global Convergence Statement for Gradient Descent with Armijo Step Size

In the optimization literature, the term *global convergence* is often used in a way that differs from the usual notion of convergence in analysis. Here, the adjective “global” indicates that the convergence statement holds for *any* initial guess $x_0 \in \mathbb{R}^n$. The term

“convergence” itself can also be somewhat misleading, since it rarely asserts that the entire sequence (x_k) produced by the algorithm converges.

Instead, a global convergence statement typically has the following form: if an optimization method, for any initial guess and for a given objective function f , always produces an iterative sequence (x_k) whose accumulation points are stationary points of f , then one refers to this property as a *global convergence* result.

Thus, while we cannot guarantee that the method produces a convergent sequence, the result is still useful. If a subsequence (x_{k_j}) converges to some $\hat{x}$, then

$$\nabla f(\hat{x}) = \mathbf{0}.$$

In other words, if the algorithm “gets stuck,” it can only do so at a stationary point. In particular, whenever the iteration happens to generate a convergent sequence, its limit is necessarily stationary. Consequently, methods such as gradient descent with Armijo step size fall within this framework of global convergence.

Algorithm 5: Gradient Descent with Armijo Step Size

```

1 Choose  $x^{(0)} \in \mathbb{R}^n$ ,  $\sigma \in (0, 1)$ ,  $\beta \in (0, 1)$ ,  $\varepsilon > 0$ .
2  $k \leftarrow 0$ .
3 while  $\|\nabla f(x^{(k)})\|_2 \geq \varepsilon$  do
4    $d^{(k)} \leftarrow -\nabla f(x^{(k)})$ .
5   Find  $t_k = \max\{\beta^\ell : \ell = 0, 1, 2, \dots\}$  such that
6      $f(x^{(k)} + t_k d^{(k)}) \leq f(x^{(k)}) + \sigma t_k \nabla f(x^{(k)})^T d^{(k)}$ .
7    $x^{(k+1)} \leftarrow x^{(k)} + t_k d^{(k)}$ .
8    $k \leftarrow k + 1$ .
```

Algorithm for Gradient Descent with Armijo Step Size The procedure is stated in [Algorithm 5](#). Before establishing a global convergence result for this method, we prove a useful lemma that can be motivated as follows. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable. For a point $x \in \mathbb{R}^n$ and a direction $d \in \mathbb{R}^n$, the directional derivative of f at x in the direction d is denoted by $f'(x; d)$ and defined as

$$f'(x; d) := \frac{\partial f}{\partial d}(x) := \lim_{t \rightarrow 0^+} \frac{f(x + td) - f(x)}{t}.$$

Since f is differentiable, this limit exists and is given explicitly by

$$f'(x; d) = \nabla f(x)^T d.$$

If f is continuously differentiable, then the following lemma shows an analogous statement in a more general situation where the fixed vectors x and d are replaced by two convergent sequences $(x^{(k)})$ and $(d^{(k)})$.

Lemma 3.3.8 Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuously differentiable and let $x, d \in \mathbb{R}^n$. Furthermore, let $(x^{(k)})$ and $(d^{(k)})$ be sequences in $\mathbb{R}^n$ such that

$$\lim_{k \rightarrow \infty} x^{(k)} = x, \quad \lim_{k \rightarrow \infty} d^{(k)} = d,$$

and let (t_k) be a sequence in $\mathbb{R}$ with

$$t_k > 0 \quad \text{for all } k, \quad \lim_{k \rightarrow \infty} t_k = 0.$$

Then

$$\lim_{k \rightarrow \infty} \frac{f(x^{(k)} + t_k d^{(k)}) - f(x^{(k)})}{t_k} = f'(x; d) = \nabla f(x)^T d.$$

Proof: Fix $k \in \mathbb{N}$ and define

$$h : [0, 1] \rightarrow \mathbb{R}^n, \quad h(\alpha) = x^{(k)} + \alpha t_k d^{(k)}.$$

Then

$$h(0) = x^{(k)}, \quad h(1) = x^{(k)} + t_k d^{(k)}.$$

By the Mean Value Theorem applied to $f \circ h$ and using the chain rule, there exists a $\xi_k \in (0, 1)$ such that

$$f(x^{(k)} + t_k d^{(k)}) - f(x^{(k)}) = (f \circ h)'(\xi_k) = \nabla f(h(\xi_k))^T h'(\xi_k).$$

Since

$$h'(\xi_k) = t_k d^{(k)},$$

we obtain

$$f(x^{(k)} + t_k d^{(k)}) - f(x^{(k)}) = t_k \nabla f(x^{(k)} + \xi_k t_k d^{(k)})^T d^{(k)}. \quad (3.5)$$

Because $0 < \xi_k < 1$ and $t_k \rightarrow 0$, the sequence (ξ_k) is bounded, and by the limit rules,

$$\lim_{k \rightarrow \infty} (x^{(k)} + \xi_k t_k d^{(k)}) = x.$$

Since f is continuously differentiable, the gradient is continuous, and therefore

$$\lim_{k \rightarrow \infty} \nabla f(x^{(k)} + \xi_k t_k d^{(k)}) = \nabla f(x).$$

Now divide the MVT identity (3.5) by t_k and take limits, which leads to

$$\lim_{k \rightarrow \infty} \frac{f(x^{(k)} + t_k d^{(k)}) - f(x^{(k)})}{t_k} = \lim_{k \rightarrow \infty} \nabla f(x^{(k)} + \xi_k t_k d^{(k)})^T d^{(k)} = \nabla f(x)^T d,$$

which is what we wanted. $\square$

With this lemma at hand, we are now ready to prove the following theorem, which provides the desired global convergence result.

Theorem 3.3.9 Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuously differentiable. Then every accumulation point of a sequence $(x^{(k)})$ generated by [Algorithm 5](#) is a stationary point of f .

Proof: Let $\hat{x}$ be an accumulation point of $(x^{(k)})$, and let $(x^{(n_k)})$ be a subsequence converging to $\hat{x}$, i.e.

$$\lim_{k \rightarrow \infty} x^{(n_k)} = \hat{x}. \quad (3.6)$$

Since ∇f is continuous and $d^{(k)} = -\nabla f(x^{(k)})$, we obtain

$$\lim_{k \rightarrow \infty} d^{(n_k)} = -\nabla f(\hat{x}). \quad (3.7)$$

Assume, for the sake of contradiction, that

$$\nabla f(\hat{x}) \neq \mathbf{0}. \quad (3.8)$$

Since $(f(x^{(k)}))$ is monotonically decreasing and $(f(x^{(n_k)}))$ converges to $f(\hat{x})$, it follows that the entire sequence $(f(x^{(k)}))$ converges to $f(\hat{x})$. Hence,

$$\lim_{k \rightarrow \infty} (f(x^{(k)}) - f(x^{(k+1)})) = 0.$$

By the choice of the search direction and the Armijo step size in [Algorithm 5](#), we have

$$f(x^{(k+1)}) - f(x^{(k)}) \leq \sigma t_k \nabla f(x^{(k)})^T d^{(k)} = -\sigma t_k \|\nabla f(x^{(k)})\|_2^2 \leq 0.$$

Therefore,

$$f(x^{(k)}) - f(x^{(k+1)}) \geq \sigma t_k \|\nabla f(x^{(k)})\|_2^2 \geq 0,$$

and by the squeeze theorem this implies

$$\lim_{k \rightarrow \infty} t_k \|\nabla f(x^{(k)})\|_2^2 = 0. \quad (3.9)$$

In particular, along the subsequence we have

$$\lim_{k \rightarrow \infty} t_{n_k} \|\nabla f(x^{(n_k)})\|_2^2 = 0.$$

Since $\nabla f(x^{(n_k)}) \rightarrow \nabla f(\hat{x}) \neq \mathbf{0}$, the limit rules for products imply that

$$\lim_{k \rightarrow \infty} t_{n_k} = 0.$$

Now write $t_{n_k} = \beta^{\ell_k}$ with $\ell_k \in \mathbb{N}_0$. By the construction of the Armijo step size, ℓ_k is chosen maximally such that t_{n_k} satisfies the Armijo condition, while

$$\theta_k := \beta^{\ell_k - 1} = \frac{t_{n_k}}{\beta} > 0$$

does not. Thus

$$f(x^{(n_k)} + \theta_k d^{(n_k)}) > f(x^{(n_k)}) + \sigma \theta_k \nabla f(x^{(n_k)})^T d^{(n_k)}.$$

Dividing by $\theta_k > 0$ yields

$$\frac{f(x^{(n_k)} + \theta_k d^{(n_k)}) - f(x^{(n_k)})}{\theta_k} > \sigma \nabla f(x^{(n_k)})^T d^{(n_k)}.$$

From (3.6), (3.7) and $\theta_k = t_{n_k}/\beta \rightarrow 0$, we can apply [Lemma 3.3.8](#) to the sequences $x^{(n_k)}$, $d^{(n_k)}$, and θ_k , and obtain

$$\lim_{k \rightarrow \infty} \frac{f(x^{(n_k)} + \theta_k d^{(n_k)}) - f(x^{(n_k)})}{\theta_k} = \nabla f(\hat{x})^T (-\nabla f(\hat{x})) = -\|\nabla f(\hat{x})\|_2^2.$$

On the other hand, taking limits in the inequality above gives

$$-\|\nabla f(\hat{x})\|_2^2 \geq \sigma \nabla f(\hat{x})^T (-\nabla f(\hat{x})) = -\sigma \|\nabla f(\hat{x})\|_2^2.$$

Since $\sigma \in (0, 1)$, this is only possible if $\nabla f(\hat{x}) = \mathbf{0}$, which contradicts (3.8). Hence every accumulation point $\hat{x}$ of $(x^{(k)})$ is stationary. $\square$

§3.4 Efficient Step Size Strategies

A *step size strategy* T assigns to each pair $(x, d) \in \mathbb{R}^n \times \mathbb{R}^n$ a subset of the positive real numbers, interpreted as the admissible step lengths for a descent step in direction d from point x . Formally,

$$T : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathcal{P}(\mathbb{R}_{>0}).$$

We call such a strategy *well-defined* if, under suitable assumptions on f , the set $T(x, d)$ is nonempty for every pair (x, d) with $\nabla f(x)^T d < 0$. [Algorithm 6](#) describes a general descent method that makes use of an arbitrary step size strategy T .

Algorithm 6: Descent Method with Step Size Strategy T

```

1 Choose  $x^{(0)} \in \mathbb{R}^n$  and  $\varepsilon > 0$ .
2  $k \leftarrow 0$ .
3 while  $\|\nabla f(x^{(k)})\|_2 \geq \varepsilon$  do
4   Choose a descent direction  $d^{(k)}$ .
5   Choose a step size  $t_k \in T(x^{(k)}, d^{(k)})$ .
6    $x^{(k+1)} \leftarrow x^{(k)} + t_k d^{(k)}$ .
7    $k \leftarrow k + 1$ .

```

Step size strategies satisfying the following property are easier to analyse in global convergence proofs than, for instance, the Armijo backtracking rule.

Definition 3.3.10 (Efficient Step Size Strategy) Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuous, let $x \in \mathbb{R}^n$, and let $d \in \mathbb{R}^n$ be a descent direction at x , i.e. $\nabla f(x)^T d < 0$. A step size strategy T is called *efficient* if there exists a constant $\theta > 0$, independent of x and d , such that

$$f(x + td) \leq f(x) - \theta \left(\frac{\nabla f(x)^T d}{\|d\|_2} \right)^2 \quad \text{for all } t \in T(x, d).$$

□

We now prove a global convergence result for the descent method.

Theorem 3.3.11 Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuously differentiable and let $(x^{(k)})_{k \in \mathbb{N}}$ be a sequence generated by [Algorithm 6](#) such that the following conditions hold:

- (a) There exists a constant $c > 0$ such that the so-called *angle condition*

$$-\frac{\nabla f(x^{(k)})^T d^{(k)}}{\|\nabla f(x^{(k)})\|_2 \|d^{(k)}\|_2} \geq c$$

holds for all k .

- (b) The step sizes $t_k > 0$ are efficient for all k .

Then every accumulation point of the sequence $(x^{(k)})$ is a stationary point.

Proof: Since each step size $t_k > 0$ is efficient, there exists a constant $\theta > 0$ such that

$$f(x^{(k+1)}) = f(x^{(k)} + t_k d^{(k)}) \leq f(x^{(k)}) - \theta \left(\frac{\nabla f(x^{(k)})^T d^{(k)}}{\|d^{(k)}\|_2} \right)^2$$

for all k . By the angle condition, it follows that

$$f(x^{(k+1)}) \leq f(x^{(k)}) - \kappa \|\nabla f(x^{(k)})\|_2^2, \quad \kappa := \theta c^2. \quad (3.10)$$

Let $\hat{x}$ be an accumulation point of $(x^{(k)})$. Since $(f(x^{(k)}))$ is monotonically decreasing and converges along a subsequence to $f(\hat{x})$, the entire sequence $(f(x^{(k)}))$ converges to $f(\hat{x})$. In particular,

$$\lim_{k \rightarrow \infty} (f(x^{(k+1)}) - f(x^{(k)})) = 0.$$

Hence, by (3.10), we immediately obtain

$$\lim_{k \rightarrow \infty} \|\nabla f(x^{(k)})\|_2 = 0.$$

Therefore, every accumulation point of the sequence $(x^{(k)})$ is a stationary point of f . $\square$

Remark 3.3.12 The angle condition is obviously satisfied for the gradient method. If the Armijo step size strategy were efficient, then by Theorem 3.3.11 the statement of Theorem 3.3.9 would follow immediately. $\square$

We will now strengthen the Armijo rule to the so-called Wolfe–Powell strategy, which is efficient provided that ∇f is Lipschitz continuous.

Theorem 3.3.13 (Wolfe–Powell Step Size Strategy) Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuously differentiable and let $\sigma \in (0, \frac{1}{2})$, $\rho \in [\sigma, 1)$, and $x^{(0)} \in \mathbb{R}^n$ be fixed. For

$$x \in \mathcal{L}(x^{(0)}) := \{z \in \mathbb{R}^n : f(z) \leq f(x^{(0)})\} \quad (\text{level set})$$

and a direction $d \in \mathbb{R}^n$ with $\nabla f(x)^T d < 0$, define

$$T_{WP}(x, d) := \left\{ t > 0 : \begin{array}{l} f(x + td) \leq f(x) + \sigma t \nabla f(x)^T d, \\ \nabla f(x + td)^T d \geq \rho \nabla f(x)^T d \end{array} \right\}$$

to be the set of Wolfe–Powell step sizes at x in direction d . Then:

- (a) If t is bounded from below, then $T_{WP}(x, d) \neq \emptyset$, i.e., the Wolfe–Powell step size strategy is well-defined.
- (b) If, in addition, the gradient ∇f is Lipschitz continuous on the level set $\mathcal{L}(x^{(0)})$, then there exists a constant $\theta > 0$ (independent of x and d) such that

$$f(x + td) \leq f(x) - \theta \left(\frac{\nabla f(x)^T d}{\|d\|_2} \right)^2$$

for all $t \in T_{WP}(x, d)$, i.e., the Wolfe–Powell step size strategy is efficient.

Remark 3.3.14 (Illustration of the Wolfe–Powell Strategy) To illustrate the Wolfe–Powell conditions, we define

$$\varphi : \mathbb{R} \rightarrow \mathbb{R}, \quad \varphi(t) := f(x + td).$$

Then the Wolfe–Powell conditions read

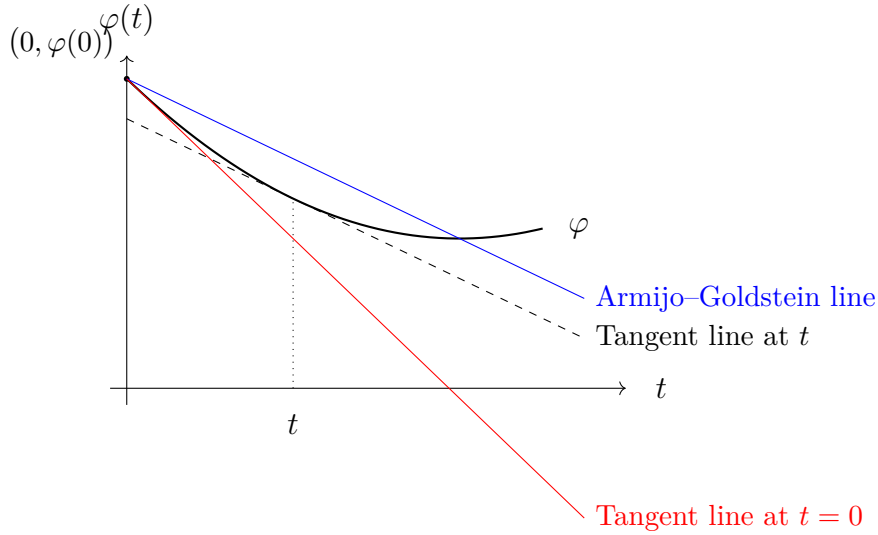
$$\varphi(t) \leq \varphi(0) + \sigma t \varphi'(0) \quad (3.11)$$

and

$$\varphi'(t) \geq \rho \varphi'(0). \quad (3.12)$$

Condition (3.11) is the Armijo condition and requires that t be chosen from a region in which the graph of φ lies below the “Armijo–Goldstein line” through $(0, \varphi(0))$.

Condition (3.12) states, roughly speaking, that t should also be chosen from a region where φ does not decrease as steeply as it does near $t = 0$. The idea is based on the observation that, near a minimizer, the slope tends to flatten out. $\square$



Remark 3.3.15 (Motivation for the strong Wolfe–Powell Strategy) In the so-called strong Wolfe–Powell strategy one additionally requires that

$$|\nabla f(x + td)^T d| \leq -\rho \nabla f(x)^T d.$$

Returning to the one-dimensional illustration of the previous remark, this replaces (3.12) with

$$|\varphi'(t)| \leq -\rho \varphi'(0). \quad (3.13)$$

The tightening consists of requiring that t be chosen from a region in which the graph of φ not only does not decrease too steeply, but also does not increase too steeply. This again reflects the idea that, near a minimizer, the derivative is expected to be small in magnitude. $\square$

§3.5 Non-linear conjugate gradient (cg) method

After addressing the determination of the step size $\alpha^{(k)}$, we will now focus on the determination of the search direction $p^{(k)}$. In particular, we describe the non-linear variant of the cg-algorithm.

We recall from Numerical Linear Algebra the (linear) cg-algorithm for the iterative numerical approximation of the solution of a linear system $Ax = a$ with a positive definite matrix A . In that case, one writes the solution $x^* = A^{-1}a$ as the minimum of the quadratic function

$$f(x) = \frac{1}{2}x^T Ax - a^T x.$$

In order to point out similarities and differences of the non-linear cg-algorithm and the linear counterpart, we repeat the cg-algorithm for the linear case, see [Algorithm 7](#). This will be done without further descriptions.

Algorithm 7: Linear cg-algorithm

Input: $x^{(0)} \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$, $a \in \mathbb{R}^n$

```

1  $r^{(0)} = Ax^{(0)} - a$ 
2  $p^{(0)} = -r^{(0)}$ 
3  $k = 0$ 
4 while  $\|r^{(k)}\| \neq 0$  do
5    $\alpha^{(k)} = \frac{(r^{(k)})^T r^{(k)}}{(p^{(k)})^T A p^{(k)}}$ 
6    $x^{(k+1)} = x^{(k)} + \alpha^{(k)} p^{(k)}$ 
7    $r^{(k+1)} = Ax^{(k+1)} - a$ 
8    $\beta^{(k+1)} = \frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}}$ 
9    $p^{(k+1)} = -r^{(k+1)} + \beta^{(k+1)} p^{(k)}$ 
10   $k \leftarrow k + 1$ 
11 end
```

Now, we are going to generalize Algorithm 7 from the specific case of a quadratic function $f(x) = \frac{1}{2}x^T Ax - a^T x$ to general smooth functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$. In order to do so, we need to take the following issues into account:

1. In the general case, the optimal step size $\alpha^{(k)}$ cannot be computed exactly and thus will be determined by one of the algorithms for the step size control introduced above.
2. In the quadratic case, the residual $r^{(k)}$ coincides with the gradient of f and can thus be evaluated exactly at the point $x^{(k)}$. In the general case, we need the gradient of the non-linear function f at $x^{(k)}$, or some appropriate approximation.

[Algorithm 8](#) shows the non-linear cg-algorithm after Fletcher–Reeves.²

Algorithm 8: Non-linear cg-algorithm after Fletcher–Reeves

Input: $x^{(0)} \in \mathbb{R}^n$, smooth $f : \mathbb{R}^n \rightarrow \mathbb{R}$

```

1 Compute  $\nabla f^{(0)} = \nabla f(x^{(0)})$ 
2  $p^{(0)} = -\nabla f^{(0)}$ 
3  $k = 0$ 
4 while  $\|\nabla f^{(k)}\| \neq 0$  do
5   Compute step size  $\alpha^{(k)}$ 
6    $x^{(k+1)} = x^{(k)} + \alpha^{(k)} p^{(k)}$ 
7   Compute  $\nabla f^{(k+1)}$ 
8    $\beta_{\text{FR}}^{(k+1)} = \frac{(\nabla f^{(k+1)})^T \nabla f^{(k+1)}}{(\nabla f^{(k)})^T \nabla f^{(k)}}$ 
9    $p^{(k+1)} = -\nabla f^{(k+1)} + \beta_{\text{FR}}^{(k+1)} p^{(k)}$ 
10   $k \leftarrow k + 1$ 
11 end
```

²Roger Fletcher (1939–2016), Colin M. Reeves (1944–).

Remark 3.3.16

(i) It is easily seen that Algorithm 8 coincides with Algorithm 7 for the special case $f(x) = \frac{1}{2}x^T Ax - a^T x$.

(ii) There are also further versions of the non-linear cg-algorithm. These essentially differ in the choice of $\beta^{(k+1)}$ in line 8. We only mention two variants:

- *Variant of Polak–Ribière*³ with

$$\beta_{\text{PR}}^{(k+1)} := \frac{\left(\nabla f^{(k+1)}\right)^T \left(\nabla f^{(k+1)} - \nabla f^{(k)}\right)}{\|\nabla f^{(k)}\|^2},$$

- *Variant of Hestenes–Stiefel*⁴ with

$$\beta_{\text{HS}}^{(k+1)} := \frac{\left(\nabla f^{(k+1)}\right)^T \left(\nabla f^{(k+1)} - \nabla f^{(k)}\right)}{\left(\nabla f^{(k+1)} - \nabla f^{(k)}\right)^T p^{(k)}}.$$

In each case, $\beta_{\text{FR}}^{(k+1)}$ in line 8 of Algorithm 8 is replaced by the corresponding alternative formula.

□

§4 Trust-Region Methods

³Elijah Polak (1931–), Gérard Ribière.

⁴Magnus Hestenes (1906–1991), Eduard Ludwig Stiefel (1909–1978).

Anhang A Supplementary Notes to Chapters

§1 Least Square

Bibliography

- [1] Horn, Roger A. und Charles R. Johnson: *Matrix Analysis*, 2. Auflage, Cambridge University Press, 2001.